

===== 文件: main.go =====

```
1|package main
2|
3|import (
4| "context"
5| "fmt"
6| "github.com/gin-gonic/gin"
7| "github.com/songzhibin97/gkit/cache/local_cache"
8| "go.uber.org/zap"
9| "server/new/app/logic/webSocket"
10|
11| //"net/http"
12| //_ "net/http/pprof" // 开启 pprof
13| "os"
14| "runtime/debug"
15| "server/Service/Ser_Init"
16| "server/api/middleware"
17| "server/core"
18| "server/global"
19| "server/new"
20| _ "server/new/app/logic"
21| "server/new/app/logic/common/setting"
22| "time"
23|)
24|
25|/*设置包自动管理*/
26|//go:generate go env -w GO111MODULE=on
27|
28|/*设置包管理下载地址*/
29|//go:generate go env -w GOPROXY=https://goproxy.cn,direct
30|
31|/*设置包自动整理信息到go.mod内*/
32|//go:generate go mod tidy
33|
34|/*设置包自动根据go.mod下载*/
35|//go:generate go mod download
36|
37|func main() {
38| //go log.Fatal(http.ListenAndServe("127.0.0.1:6060", nil))
39| defer func() {
40| if err := recover(); err != nil {
41| 局_上报错误 := fmt.Sprintf("全局捕获错误:\n", err, "\n堆栈信息:\n", string(debug.Stack()))
42| fmt.Println("发生致命错误:", 局_上报错误)
43| global.Q快验.Z置新用户消息(2, 局_上报错误)
44|
45| }
46| }()
47|
48| global.GVA_Viper = core.InitViper() //初始化配置读写器 和全局配置结构变量GVA_config
49|
50| global.GVA_LOG = core.InitZap() // 初始化zap日志记录器
51| zap.ReplaceGlobals(global.GVA_LOG) //替换系统的log记录器 为zap的全局日志记录器 方便统一管理
52| global.H缓存 = local_cache.NewCache( //需要比数据库先初始化,因为读取系统配置就用到缓存了
53| //设置缓存默认超时时间 为24小时
54| local_cache.SetDefaultExpire(time.Hour*24),
55| local_cache.SetCapture(func(k string, v interface{}) { //设置缓存删除捕获函数,缓存到期删除时会触发
56| //fmt.Printf("delete k:%s v:%v\n", k, v)
57| })),
58| )
```

```

59| global.GVA_DB, _ = Ser_Init.InitGormMysql() // gorm连接数据库
Gorm参考资料https://www.cnblogs.com/davis12/p/16365213.html
60|
61| if global.GVA_DB != nil { //如果数据库不为空
62| c := gin.Context{}
63| Ser_Init.InitDbTables(&c) // 如果数据库连接成功就初始化表 //不在这了,只能由 InitMysql 初始化
64|
65| // 程序结束前关闭数据库链接
66| db, _ := global.GVA_DB.DB()
67| defer db.Close() //延迟关闭程序结束前关闭表
68| middleware.J集_UserAPI路由_加密.G更新md5API名称(setting.Q系统设置().Y用户API加密盐) //只有数据库成功才可以操作
不然或报错
69| websocket.L_webSocket.D断开所有连接()
70| } else {
71| global.GVA_LOG.Info(fmt.Sprintf("数据库连接失败,等待输入数据库信息"))
72| }
73| core.InitCron定时任务()
74| new.Main()
75|
76| core.RunWindowsServer() //启动web服务器 先启动 不然无法自验证
77|
78|}
79|
80|func STOP() {
81| global.Cron定时任务.Cron.Stop()
82| //先关闭端口 解除占用
83| global.GVA_Gin.Shutdown(context.Background()) //这句话可以停止侦听关闭端口
84| // 退出当前进程
85| os.Exit(0)
86|
87|}
===== 文件: new\main.go =====
1|package new
2|
3|import (
4| _ "server/new/app/init"
5|)
6|
7|func Main() {
8|
9|}
===== 文件: new\test\main.go =====
1|package main
2|
3|func main() {
4| test()
5|}
===== 文件: KuaiYanSDK\aliyun.go =====
1|package KuaiYanSDK
2|
3|import (
4| "bytes"
5| "context"
6| "crypto/hmac"
7| "crypto/md5"
8| "crypto/sha256"
9| "encoding/base64"
10| "fmt"
11| "github.com/google/uuid"
12| "github.com/imroc/req/v3"

```

```

13| "io/ioutil"
14| "net/http"
15| "sort"
16| "strconv"
17| "strings"
18| "time"
19|)
20|
21|// HTTP header keys.
22|const (
23| HTTPHeaderAccept = "Accept"
24| HTTPHeaderContentMD5 = "Content-MD5"
25| HTTPHeaderContentType = "Content-Type"
26| HTTPHeaderDate = "Date"
27| HTTPHeaderUserAgent = "User-Agent"
28|)
29|
30|// HTTP header keys used for Aliyun API gateway signature.
31|const (
32| HTTPHeaderCAPrefix = "X-Ca-"
33| HTTPHeaderCAKey = "X-Ca-Key"
34| HTTPHeaderCANonce = "X-Ca-Nonce"
35| HTTPHeaderCASignature = "X-Ca-Signature"
36| HTTPHeaderCASignatureHeaders = "X-Ca-Signature-Headers"
37| HTTPHeaderCASignatureMethod = "X-Ca-Signature-Method"
38| HTTPHeaderCATimestamp = "X-Ca-Timestamp"
39|)
40|
41|// HTTP header content-type values.
42|const (
43| HTTPContentTypeForm = "application/x-www-form-urlencoded"
44| HTTPContentTypeMultipartForm = "multipart/form-data"
45| HTTPContentTypeMultipartFormWithBoundary = "multipart/form-data; boundary="
46| HTTPContentTypeStream = "application/octet-stream"
47| HTTPContentTypeJson = "application/json"
48| HTTPContentTypeXml = "application/xml"
49| HTTPContentTypeText = "text/plain"
50|)
51|
52|// HTTP method values.
53|const (
54| HTTPMethodGet = "GET"
55| HTTPMethodPost = "POST"
56| HTTPMethodPut = "PUT"
57| HTTPMethodDelete = "DELETE"
58| HTTPMethodPatch = "PATCH"
59| HTTPMethodHead = "HEAD"
60| HTTPMethodOptions = "OPTIONS"
61|)
62|
63|// default values.
64|const (
65| defaultUserAgent = "Go-Aliyun-Sign-Client"
66| defaultAccept = "*/*"
67| defaultSignMethod = "HmacSHA256"
68| defaultLF = "\n"
69| defaultSep = ","
70|)
71|
72|var signHeaders = map[string]struct{}{

```

```

73| http.CanonicalHeaderKey(HTTPHeaderCAKey): {},
74| http.CanonicalHeaderKey(HTTPHeaderCANonce): {},
75| http.CanonicalHeaderKey(HTTPHeaderCASignatureMethod): {},
76| http.CanonicalHeaderKey(HTTPHeaderCATimestamp): {},
77|}
78|
79|func getSortKeys(m map[string][]string, needFormat ...bool) []string {
80| nf := false
81| if len(needFormat) > 0 {
82| nf = needFormat[0]
83| }
84|
85| keys := make([]string, 0, len(m))
86| for key := range m {
87| if nf {
88| key = http.CanonicalHeaderKey(key)
89| }
90| keys = append(keys, key)
91| }
92|
93| sort.Strings(keys)
94|
95| return keys
96|}
97|
98|// copyRequest returns the copied request.
99|func copyRequest(r *http.Request) (*http.Request, error) {
100| clone := r.Clone(context.Background())
101| if r.Body == nil {
102| clone.Body = nil
103| } else if r.Body == http.NoBody {
104| clone.Body = http.NoBody
105| } else if r.GetBody != nil {
106| body, err := r.GetBody()
107| if err != nil {
108| return nil, fmt.Errorf("request get body err: %w", err)
109| }
110|
111| clone.Body = body
112| } else {
113| var buf bytes.Buffer
114| if _, err := buf.ReadFrom(r.Body); err != nil {
115| return nil, fmt.Errorf("read from request body err: %w", err)
116| }
117| if err := r.Body.Close(); err != nil {
118| return nil, fmt.Errorf("request body close err: %w", err)
119| }
120|
121| r.Body = ioutil.NopCloser(&buf)
122| clone.Body = ioutil.NopCloser(bytes.NewBuffer(buf.Bytes()))
123| }
124|
125| return clone, nil
126|}
127|
128|// CurrentTimeMillis returns the millisecond representation of the current time.
129|func CurrentTimeMillis() string {
130| t := time.Now().UnixNano() / 1000000
131|
132| return strconv.FormatInt(t, 10)

```

```

133|}
134|
135|// CurrentGMTDate returns the GMT date representation of the current time.
136|func CurrentGMTDate() string {
137| return time.Now().UTC().Format(http.TimeFormat)
138|}
139|
140|// UUID4 returns random generated UUID string.
141|func UUID4() string {
142| u, err := uuid.NewRandom()
143| if err != nil {
144| return ""
145| }
146|
147| return u.String()
148|}
149|
150|// HmacSHA256 returns the string encrypted with HmacSHA256 method.
151|func HmacSHA256(b, key []byte) string {
152| h := hmac.New(sha256.New, key)
153| h.Write(b)
154|
155| return base64.StdEncoding.EncodeToString(h.Sum(nil))
156|}
157|
158|// MD5 returns the string hashed with MD5 method.
159|func MD5(b []byte) string {
160| m := md5.New()
161| m.Write(b)
162|
163| return base64.StdEncoding.EncodeToString(m.Sum(nil))
164|}
165|
166|// Sign will sign the request with appKey and appKeySecret.
167|func aliyunSign(req *req.Request, appKeySecret, appKey string) error {
168| req.SetHeader(HTTPHeaderCAKey, appKey)
169| req.SetHeader(HTTPHeaderCANonce, UUID4())
170| req.SetHeader(HTTPHeaderCASignatureMethod, defaultSignMethod)
171| req.SetHeader(HTTPHeaderCATimestamp, CurrentTimeMillis())
172|
173| if req.Headers.Get(HTTPHeaderAccept) == "" {
174| req.SetHeader(HTTPHeaderAccept, defaultAccept)
175| }
176|
177| if req.Headers.Get(HTTPHeaderDate) == "" {
178| req.SetHeader(HTTPHeaderDate, CurrentGMTDate())
179| }
180|
181| if req.Headers.Get(HTTPHeaderUserAgent) == "" {
182| req.SetHeader(HTTPHeaderUserAgent, defaultUserAgent)
183| }
184|
185| ct := req.Headers.Get(HTTPHeaderContentType)
186| if req.Body != nil && ct != HTTPContentTypeForm &&
187| !strings.HasPrefix(ct, HTTPContentTypeMultipartFormWithBoundary) {
188| req.SetHeader(HTTPHeaderContentMD5, MD5(req.Body))
189| }
190|
191| stringToSign, err := buildStringToSign(req)
192| if err != nil {

```

```

193| return fmt.Errorf("build string to sign err: %w", err)
194| }
195|
196| req.SetHeader(HTTPHeaderCASignature, HmacSHA256([]byte(stringToSign), []byte(appKeySecret)))
197|
198| return nil
199|}
200|
201|func buildStringToSign(req *req.Request) (string, error) {
202| var s strings.Builder
203| s.WriteString(strings.ToUpper(req.Method) + defaultLF)
204|
205| s.WriteString(req.Headers.Get(HTTPHeaderAccept) + defaultLF)
206| s.WriteString(req.Headers.Get(HTTPHeaderContentMD5) + defaultLF)
207| s.WriteString(req.Headers.Get(HTTPHeaderContentType) + defaultLF)
208| s.WriteString(req.Headers.Get(HTTPHeaderDate) + defaultLF)
209| s.WriteString(buildHeaderStringToSign(req))
210| s.WriteString(req.URL.Path + "?" + req.URL.RawQuery)
211|
212| return s.String(), nil
213|}
214|
215|func buildHeaderStringToSign(req *req.Request) string {
216| var builder strings.Builder
217| signHeaderKeys := make([]string, 0)
218| headerKeys := getSortKeys(req.Headers, true)
219|
220| for _, key := range headerKeys {
221| if _, ok := signHeaders[key]; ok {
222| signHeaderKeys = append(signHeaderKeys, key)
223| builder.WriteString(key + ":" + req.Headers.Get(key) + defaultLF)
224| }
225| }
226|
227| req.SetHeader(HTTPHeaderCASignatureHeaders, strings.Join(signHeaderKeys, defaultSep))
228|
229| return builder.String()
230|}
==== 文件: KuaiYanSDK\工具函数.go ====
1|package KuaiYanSDK
2|
3|import (
4| "crypto"
5| "crypto/hmac"
6| "crypto/md5"
7| "crypto/rand"
8| "crypto/rsa"
9| "crypto/sha1"
10| "encoding/base64"
11| "encoding/hex"
12| "encoding/json"
13| "errors"
14| "fmt"
15| "github.com/imroc/req/v3"
16| "github.com/valyala/fastjson"
17| "log"
18| "math/big"
19| "net/url"
20| "server/utils"
21| "sort"

```

```

22| "strings"
23| "time"
24|)
25|
26|func (k *Api快验_类) 通讯(postJson map[string]interface{}) (*fastjson.Value, bool) {
27| 局_文本 := k.发包并返回解密(k.加密并签名(postJson))
28|  //fmt.Printf("响应明文数据:%s\n", 局_文本)
29|  if 局_文本 == "" {
30|    // 直接返回即可,错误原因 在 发包并返回解密 已经有了
31|    return nil, false
32|  }
33|
34|  响应, err := fastjson.Parse(局_文本)
35|
36|  if err != nil || !k.X响应校验时间状态(postJson, 响应) { //如果结果解析失败,或时间,状态码错误,直接返回假
37|    return nil, false
38|  }
39|  return 响应, true
40|}
41|
42|/*
43| */
44|func (k *Api快验_类) X响应校验时间状态(请求 map[string]interface{}, 响应 *fastjson.Value) bool {
45|
46|  if 请求["Status"].(int64) != 响应.GetInt64("Status") {
47|    k.集_错误代码 = 响应.GetInt("Status")
48|    k.集_错误信息 = string(响应.GetStringBytes("Msg"))
49|    return false
50|  }
51|  局_耗时 := 请求["Time"].(int64) - 响应.GetInt64("Time")
52|
53|  if 局_耗时 > 1800 { // ' 发包和收包时间超过30分钟,就有点不对了吧,和服务器时间再差也不会差这么多
54|    if string(响应.GetStringBytes("Msg")) == "" {
55|      k.集_错误信息 = "封包时间异常"
56|      k.集_错误代码 = 107
57|    } else {
58|      k.集_错误代码 = 响应.GetInt("Status")
59|      k.集_错误信息 = string(响应.GetStringBytes("Msg"))
60|    }
61|  }
62|  return true
63|}
64|
65|func (k *Api快验_类) 加密并签名(postJson map[string]interface{}) string {
66|  //添加公共变量
67|  postJson["Time"] = time.Now().Unix()
68|  局_随机数, _ := rand.Int(rand.Reader, big.NewInt(89999))
69|  postJson["Status"] = 局_随机数.Int64() + 10000
70|  局_Api := postJson["Api"].(string) // 如果没有Api 会报恐慌
71|
72|  // 验证码id可能会缓存,所以必须是有验证码值时才携带 ,并清除缓存
73|  if k.集_验证码值 != "" && k.集_验证码ID != "" {
74|    postJson["Captcha"] = Captcha{
75|      Type: k.集_验证码类型,
76|      Id: k.集_验证码ID,
77|      Value: k.集_验证码值,
78|    }
79|    k.集_验证码类型 = 0
80|    k.集_验证码ID = ""
81|    k.集_验证码值 = ""

```

```

82| }
83| 局_明文, _ := json.Marshal(postJson)
84| //fmt.Printf("发送明文数据:" + string(局_明文))
85| if k.集_CryptoType == 1 {
86| // 都明文了,还加密什么,签名什么啊 直接返回就好了
87| return string(局_明文)
88| }
89|
90| var 局_密文, 局_签名 string
91| if k.集_CryptoType == 3 && strings.Contains(强制Rsa加密接口, ``+局_Api+``) {
92| /* 看看接口是否在强制RSA通讯列表里,重要包RSA通讯,保证安全,不重要封包Aes通讯,保证服务器性能,
93| ' 不过即使使用了RSA加密不重要的包,也是可以解开的,服务器会自动判断,但是应该RSA通讯的封包使用了AES通讯服务器会风控报
错加解密方式错误
94| ' 简单来说就是封包安全可以向上兼容,不重要的封包可以使用更安全的RSA通讯方式,但是重要封包不能使用不安全AES的通讯方式*/
95| //RSA 就是随机AES密钥 然后RSA公钥加密放到签名里
96| 局_随机AES密钥 := make([]byte, 24)
97| rand.Read(局_随机AES密钥)
98| // #对称算法 AES_192_CBC, #数据填充_PKCS7_PADDING, iv取空白字节集 (24)
99| 局_临时字节集 := utils.Aes加密_cbc192密钥字节数组(string(局_明文), 局_随机AES密钥)
100| 局_密文 = base64.StdEncoding.EncodeToString(局_临时字节集)
101| //公钥加密aes密钥,
102| 局_签名 = k.rsa公钥加密(局_随机AES密钥)
103| } else {
104| //非明文 非 rsa 通用加密
105| // 正常aes加密即可
106| 局_临时字节集 := utils.Aes加密_cbc192密钥字节数组(string(局_明文), k.J_CryptoKeyAes)
107| 局_密文 = base64.StdEncoding.EncodeToString(局_临时字节集)
108| // 注意不是md5(明文) 是base64密文文本的md5 //base64 可以防止gbkUtf8编码不同导致的签名不正确
109| 局_签名 = utils.Md5String(局_密文 + string(k.J_CryptoKeyAes))
110| }
111| //fmt.Printf(`{"a": "%s", "b": "%s"}`, 局_密文, 局_签名)
112| return fmt.Sprintf(`{"a": "%s", "b": "%s"}`, 局_密文, 局_签名)
113|}
114|
115|type Captcha struct {
116| Type int `json:"Type"`
117| Id string `json:"Id"`
118| Value string `json:"Value"`
119|}
120|type Result struct {
121| Data string `json:"data"`
122|}
123|
124|func (k *Api快验_类) 发包并返回解密(post内容 string) string {
125| var 局_返回 string
126| //耗时 := time.Now().UnixMilli()
127| client := req.C().SetTimeout(60 * time.Second).EnableInsecureSkipVerify() //DevMode()
128|
129| //网管 应用认证
130| //client.WrapRoundTripFunc(k.腾讯云Api网关认证中间件)
131| client.WrapRoundTripFunc(k.阿里云Api网关认证中间件)
132| //循环3次 容错处理
133| for i := 0; i < 3; i++ {
134| resp, _ := client.R().
135| SetHeader("Token", k.J_Token).
136| SetBodyString(post内容).
137| Post(k.集_AppWeb)
138| 局_返回 = resp.String()
139| if 局_返回 != "" {
140| //不为空就跳出

```

```

141| break
142| }
143| fmt.Printf("Time:%s:通讯异常容错计次:%d\n", time.Now().String(), l)
144| time.Sleep(2 * time.Second) // 延迟2秒在重试
145| }
146| //fmt.Printf("通讯耗时:%d\n", time.Now().UnixMilli()-耗时)
147| if 局_返回 == "" {
148| k.集_错误信息 = "请求失败,可能以下原因,电脑时间不准确或无网络连接, 请校准时间后尝试重启软件."
149| k.集_错误代码 = 0
150| return ""
151| }
152|
153| 响应, err := fastjson.Parse(局_返回)
154| if err != nil { // 解析失败,直接按错误处理,可能是404 服务器中间件不通之类的
155| k.集_错误信息 = 局_返回
156| k.集_错误代码 = 0
157| return ""
158| }
159|
160| if 响应.GetInt("Time") > 0 { //明文直接返回
161| return 局_返回
162| }
163| 局_密文 := string(响应.GetStringBytes("a"))
164| var 局_AES密匙 []byte
165| if k.集_CryptoType == 3 {
166| 局_签名 := string(响应.GetStringBytes("b"))
167| if len(局_签名) > 32 { //如果长度大于32 那就不是签名而是密文AES加密密匙,需要通过RSA公钥解密
168| 局_AES密匙, err = base64.StdEncoding.DecodeString(局_签名)
169| if err != nil {
170| k.集_错误信息 = "RSA解密失败"
171| k.集_错误代码 = 0
172| return ""
173| }
174| 局_AES密匙 = k.RSA公钥解密(局_AES密匙)
175| } else {
176| if strings.ToUpper(utils.Md5String(局_密文+string(k.J_CryptoKeyAes))) != strings.ToUpper(局_签名) {
177| k.集_错误信息 = "验签不通过"
178| k.集_错误代码 = 0
179| return ""
180| }
181| 局_AES密匙 = k.J_CryptoKeyAes
182| }
183| } else {
184| 局_AES密匙 = k.J_CryptoKeyAes
185| }
186| return utils.Aes解密_cbc192字节集(编码_base64解码(局_密文), 局_AES密匙)
187|}
188|
189|func (k *Api快验_类) rsa公钥加密(加密内容 []byte) (base64密文 string) {
190| if k.集_公钥指针 == nil {
191| return ""
192| }
193| //4、公钥加密
194| cipherData, err := rsa.EncryptPKCS1v15(rand.Reader, k.集_公钥指针, 加密内容)
195| if err != nil {
196| return ""
197| }
198| base64密文 = base64.StdEncoding.EncodeToString(cipherData)
199| return base64密文
200|}

```

```

201|}
202|
203|func (k *Api快验_类) RSA公钥解密(密文 []byte) (明文字节集 []byte) {
204|
205| 明文字节集, err := publicDecrypt(k.集_公钥指针, crypto.Hash(0), nil, 密文)
206|  if err != nil {
207|    return 明文字节集
208|  }
209|  return 明文字节集
210|}
211|
212|func 编码_base64编码(字节集 []byte) string {
213|  return base64.StdEncoding.EncodeToString(字节集)
214|}
215|}
216|func 编码_base64解码(字符串 string) []byte {
217|  局_临时字节集, err := base64.StdEncoding.DecodeString(字符串)
218|  if err == nil {
219|    return 局_临时字节集
220|  }
221|  return []byte{}
222|}
223|
224|// copy&modified from crypt/rsa/pkcs1v5.go
225|func publicDecrypt(pub *rsa.PublicKey, hash crypto.Hash, hashed []byte, sig []byte) (out []byte, err error) {
226|  hashLen, prefix, err := pkcs1v15HashInfo(hash, len(hashed))
227|  if err != nil {
228|    return nil, err
229|  }
230|
231|  tLen := len(prefix) + hashLen
232|  k := (pub.N.BitLen() + 7) / 8
233|  if k < tLen+11 {
234|    return nil, fmt.Errorf("length illegal")
235|  }
236|
237|  c := new(big.Int).SetBytes(sig)
238|  m := encrypt(new(big.Int), pub, c)
239|  em := leftPad(m.Bytes(), k)
240|  out = unLeftPad(em)
241|
242|  err = nil
243|  return
244|}
245|
246|// copy from crypt/rsa/pkcs1v5.go
247|func pkcs1v15HashInfo(hash crypto.Hash, inLen int) (hashLen int, prefix []byte, err error) {
248|  // Special case: crypto.Hash(0) is used to indicate that the data is
249|  // signed directly.
250|  if hash == 0 {
251|    return inLen, nil, nil
252|  }
253|
254|  hashLen = hash.Size()
255|  if inLen != hashLen {
256|    return 0, nil, errors.New("crypto/rsa: input must be hashed message")
257|  }
258|  prefix, ok := hashPrefixes[hash]
259|  if !ok {
260|    return 0, nil, errors.New("crypto/rsa: unsupported hash function")

```

```

261| }
262| return
263|}
264|
265|// copy from crypt/rsa/pkcs1v5.go
266|func leftPad(input []byte, size int) (out []byte) {
267| n := len(input)
268| if n > size {
269| n = size
270| }
271| out = make([]byte, size)
272| copy(out[len(out)-n:], input)
273| return
274|}
275|func unLeftPad(input []byte) (out []byte) {
276| n := len(input)
277| t := 2
278| for i := 2; i < n; i++ {
279| if input[i] == 0xff {
280| t = t + 1
281| } else {
282| if input[i] == input[0] {
283| t = t + int(input[1])
284| }
285| break
286| }
287| }
288| out = make([]byte, n-t)
289| copy(out, input[t:])
290| return
291|}
292|
293|// copy from crypt/rsa/pkcs1v5.go
294|var hashPrefixes = map[crypto.Hash][]byte{
295| crypto.MD5: {0x30, 0x20, 0x30, 0x0c, 0x06, 0x08, 0x2a, 0x86, 0x48, 0x86, 0xf7, 0x0d, 0x02, 0x05, 0x05, 0x00, 0x04, 0x10},
296| crypto.SHA1: {0x30, 0x21, 0x30, 0x09, 0x06, 0x05, 0x2b, 0x0e, 0x03, 0x02, 0x1a, 0x05, 0x00, 0x04, 0x14},
297| crypto.SHA224: {0x30, 0x2d, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x04, 0x05, 0x00, 0x04, 0x1c},
298| crypto.SHA256: {0x30, 0x31, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x01, 0x05, 0x00, 0x04, 0x20},
299| crypto.SHA384: {0x30, 0x41, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x02, 0x05, 0x00, 0x04, 0x30},
300| crypto.SHA512: {0x30, 0x51, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x03, 0x05, 0x00, 0x04, 0x40},
301| crypto.MD5SHA1: {}, // A special TLS case which doesn't use an ASN1 prefix.
302| crypto.RIPMD160: {0x30, 0x20, 0x30, 0x08, 0x06, 0x06, 0x28, 0xcf, 0x06, 0x03, 0x00, 0x31, 0x04, 0x14},
303|}
304|
305|// copy from crypt/rsa/pkcs1v5.go
306|func encrypt(c *big.Int, pub *rsa.PublicKey, m *big.Int) *big.Int {
307| e := big.NewInt(int64(pub.E))
308| c.Exp(m, e, pub.N)
309| return c
310|}
311|func (k *Api快验 类) 腾讯云Api网关认证中间件(rt req.RoundTripper) req.RoundTripFunc {
312| return func(req *req.Request) (resp *req.Response, err error) {
313| if len(k.集_Api网关ApiAppSecret) == 0 || k.集_Api网关ApiAppKey == "" {
314| resp, err = rt.RoundTrip(req)
315| return
316| }

```

```

317| // before request
318| // 应用 ApiAppKey
319| //const ApiAppKey = "APID3uzfbf4phbgshae2yMg04h1W0s7BpjXqF646"
320| //应用 ApiAppSecret
321| //const ApiAppSecret = "72Jg5ODOv1sRvskQZde99I3nKr4hf33y6529Qsc"
322|
323| const Accept = "application/json"
324| const ContentType = "application/json"
325|
326| req.Headers.Set("Source", "RSA2")
327| req.Headers.Set("Accept", Accept)
328| if req.Headers.Get("Token") == "" { //防止没有这个参数
329| req.Headers.Set("Token", "")
330| }
331|
332| // 根据 Url 解析 Host 和 Path
333| u, err := url.Parse(req.URL.String())
334| if err != nil {
335| log.Fatal(err)
336| }
337| Host := u.Hostname()
338| Path := u.Path
339| Query := u.RawQuery
340|
341| // 签名path不带环境信息
342| if strings.HasPrefix(Path, "/release") {
343| Path = strings.TrimPrefix(Path, "/release")
344| } else if strings.HasPrefix(Path, "/test") {
345| Path = strings.TrimPrefix(Path, "/test")
346| } else if strings.HasPrefix(Path, "/prepub") {
347| Path = strings.TrimPrefix(Path, "/prepub")
348| }
349|
350| if Path == "" {
351| Path = "/"
352| }
353|
354| // 拼接query参数, query参数需要按字典序排序
355| if len(Query) > 0 {
356| args, _ := url.ParseQuery(Query)
357| var keys []string
358| for c := range args {
359| keys = append(keys, c)
360| }
361| sort.Strings(keys)
362| sortQuery := ""
363| for _, c := range keys {
364| if args[c][0] != "" {
365| sortQuery = sortQuery + "&" + c + "=" + args[c][0]
366| } else {
367| sortQuery = sortQuery + "&" + c
368| }
369| }
370| sortQuery = strings.TrimPrefix(sortQuery, "&")
371|
372| Path = Path + "?" + sortQuery
373| }
374|
375| // 获取当前 UTC
376| xDate := time.Now().UTC().Format("Mon, 02 Jan 2006 15:04:05 GMT")

```

```

377| ContentMD5 := ""
378| if req.Method == "POST" {
379| h := md5.New()
380| h.Write(req.Body)
381| md5Str := hex.EncodeToString(h.Sum(nil))
382| ContentMD5 = base64.StdEncoding.EncodeToString([]byte(md5Str))
383| }
384|
385| // 构造签名
386| signingStr := fmt.Sprintf("source: %s\ntoken: %s\nx-date: %s\n%s\n%s\n%s\n%s", req.Headers.Get("Source"),
req.Headers.Get("Token"), xDate, req.Method, Accept, ContentType,
387| ContentMD5, Path)
388| mac := hmac.New(sha1.New, k.集_Api网关ApiAppSecret)
389|
390| _, err = mac.Write([]byte(signingStr))
391| if err != nil {
392| log.Fatal(err)
393| }
394| signature := base64.StdEncoding.EncodeToString(mac.Sum(nil))
395| sign := fmt.Sprintf("hmac id=\"%s\", algorithm=\"%s-hmac-sha1\", headers=\"%s source token x-date\", signature=\"%s\"",
396| k.集_Api网关ApiAppKey, signature)
397|
398| req.Headers.Set("Host", Host)
399| req.Headers.Set("Accept", Accept)
400| req.Headers.Set("Content-Type", ContentType)
401| req.Headers.Set("x-date", xDate)
402| req.Headers.Set("Authorization", sign)
403| // 构造请求
404|
405| resp, err = rt.RoundTrip(req)
406| // after response
407| // ...
408| return
409| }
410|}
411|func (k *Api快验_类) 阿里云Api网关认证中间件(rt req.RoundTripper) req.RoundTripFunc {
412| return func(req *req.Request) (resp *req.Response, err error) {
413| if len(k.集_Api网关ApiAppSecret) == 0 || k.集_Api网关ApiAppKey == "" {
414| resp, err = rt.RoundTrip(req)
415| return
416| }
417|
418| _ = aliyunSign(req, k.集_Api网关ApiAppKey, string(k.集_Api网关ApiAppSecret))
419|
420| // 构造请求
421|
422| resp, err = rt.RoundTrip(req)
423| // after response
424| // ...
425| return
426| }
427|}
===== 文件: KuaiYanSDK\快验对接.go =====
1|// Package KuaiYanSDK 飞鸟快验go语言对接Sdk
2|package KuaiYanSDK
3|
4|import (
5| "EFunc/utils"
6| "crypto/rsa"
7| "crypto/x509"

```

```

8| "encoding/pem"
9| "fmt"
10| "github.com/gogf/gf/v2/encoding/gjson"
11| "github.com/valyala/fastjson"
12| "runtime"
13|)
14|
15|const 强制Rsa加密接口 = ` "GetToken": 1,
16| "UserLogin": 1,
17| "UserReduceMoney": 1,
18| "UserReduceVipNumber": 1,
19| "UserReduceVipTime": 1,
20| "GetVipData": 1,`
21|
22|type Api快验_类 struct {
23| 集_AppWeb, J_Token, 集_错误信息, 集_验证码ID, 集_验证码值 string
24| 集_CryptoKeyAes []byte //通讯Aes密钥
25| 集_CryptoType int //1 明文 2 MD5签名AES加密 3 Rsa签名交换AES密钥
26| 集_错误代码 int
27| 集_公钥指针 *rsa.PublicKey
28| 集_验证码类型 int
29| 集_Api网关ApiAppKey string
30| 集_Api网关ApiAppSecret []byte
31| 集_连接方式 int
32|}
33|
34|func (k *Api快验_类) SetAppWeb(域名 string) bool {
35| k.集_AppWeb = 域名 + string(utils.B编码_BASE64解码("L0FwaT9BcHBjZD0xMDAwMQ=="))
36| return true
37|}
38|
39|// 配置json 可以直接在应用设置里复制
40|func (k *Api快验_类) C初始化配置(配置json string) bool {
41| 局_fastjson, jsonErr := fastjson.Parse(配置json)
42| if jsonErr != nil {
43| k.集_错误信息 = "配置json解析失败"
44| return false
45| }
46| //fmt.Printf(string(局_fastjson.GetInt("CryptoType")))
47| k.集_CryptoType = 局_fastjson.GetInt("CryptoType")
48| k.集_AppWeb = string(局_fastjson.GetStringBytes("AppWeb"))
49| path := utils.C程序_取运行目录()
50| if runtime.GOOS == "windows" {
51| path = "."
52| }
53| path = path + "/config.json" //设置文件目录 //注意设置 ./config.json 宝塔写文件不会写运行目录 文件会在
/www/server/panel 文件夹
54|
55| if gjson.New(utils.W文件_读入文本(path)).Get("系统模式").Int64() == 1056795985 {
56| k.SetAppWeb("http://127.0.0.1:18888")
57| fmt.Printf("超级管理员模式\n")
58| }
59| k.集_CryptoKeyAes = 局_fastjson.GetStringBytes("CryptoKeyAes")
60| k.集_Api网关ApiAppSecret = []byte("204641349")
61| k.集_Api网关ApiAppKey = "gc0Aay7WvmO8X5tzlwTupEVsQ9TXlmJz"
62|
63| switch k.集_CryptoType {
64| case 3:
65| k.集_CryptoKeyAes = []byte(utils.W文本_取随机字符串(24))
66| //2、 pem decode,得到block的der编码数据

```

```

67| block, _ := pem.Decode(局_fastjson.GetStringBytes("CryptoKeyPublic"))
68| if block == nil {
69| k.集_错误信息 = "Rsa密钥格式错误"
70| return false
71| }
72| //3、解码der得到公钥
73| //publicKey, err := x509.ParsePKCS1PublicKey(derText) // 这个只能载入PKCS1的 但是目前公钥基本都是PKCS8的
74| publicKeyInterface, err := x509.ParsePKIXPublicKey(block.Bytes) //这个只能载入PKCS8的公钥 //大坑 必须校验是否
block是否为nil 否则block.Bytes卡主,类似进入许可区,
75| if err != nil {
76| k.集_错误信息 = "Rsa密钥错误"
77| return false
78| //密钥错误
79| }
80| //类型断言
81| ok := false
82| k.集_公钥指针, ok = publicKeyInterface.(*rsa.PublicKey) //强制转换
83| if !ok {
84| k.集_错误信息 = "Rsa密钥错误"
85| }
86| return ok
87| case 2:
88| k.集_错误信息 = "Aes密钥错误"
89| return len(k.J_CryptoKeyAes) == 24
90| }
91|
92| return k.集_CryptoType &gt; 0
93|
94|}
95|
96|func (k *Api快验_类) Q取错误信息(存储错误代号的变量 *int) string {
97|
98| if 存储错误代号的变量 != nil {
99| *存储错误代号的变量 = k.集_错误代码
100| }
101| return k.集_错误信息
102|}
103|
104|// Z置验证码信息
105|// .子程序 置验证码信息, 文本型, 公开, 需要验证码的接口,前一句代码置入验证码值,接口就会携带提交
106|// .参数 验证码类型, 整数型, 可空, 空 不覆盖缓存 1 英数验证码,2 行为验证码,3 短信验证码
107|// .参数 验证码ID, 文本型, 可空, 空 不覆盖缓存 可分两次先置id,验证码类型 后置值
108|// .参数 验证码值, 文本型, 可空, 空 不覆盖缓存/*
109|func (k *Api快验_类) Z置验证码信息(验证码类型 int, 验证码ID, 验证码值 string) {
110| if 验证码类型 &gt; 0 {
111| k.集_验证码类型 = 验证码类型
112| }
113| if 验证码ID != "" {
114| k.集_验证码ID = 验证码ID
115| }
116| if 验证码值 != "" {
117| k.集_验证码值 = 验证码值
118| }
119|}
120|
121|/*
122|.子程序 取Token, 逻辑型, 公开, 初始化后 首先调用,获取token标志
123|*/
124|func (k *Api快验_类) Q取Token() bool {
125| 请求json := make(map[string]interface{}, 5)

```

```

126| 请求json["Api"] = "GetToken"
127| if len(k.J_CryptoKeyAes) != 24 { //可能有意外导致的删除Aes,比如读取上次重启前的token和AesKey
    所以重新获取的时候,需要重置
128| k.J_CryptoKeyAes = []byte(utils.W文本_取随机字符串(24))
129| }
130| if k.集_CryptoType == 3 {
131| 请求json["Key"] = string(k.J_CryptoKeyAes)
132| }
133| k.J_Token = ""
134| 响应json, ok := k.通讯(请求json)
135| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
136| return false
137| }
138|
139| // {"Data":{"Token":"ALYVZWFRDF7ED72VLZMU2Q8CEHFFUKJP","CryptoKeyAes":"APfsSNcyziMBa36CTRcEZGbk","IP":"1
    27.0.0.1"},"Time":1688007301,"Status":84661,"Note":""}
140| k.J_Token = string(响应json.GetStringBytes("Data", "Token"))
141| if k.集_CryptoType == 3 {
142| k.J_CryptoKeyAes = 响应json.GetStringBytes("Data", "CryptoKeyAes")
143| }
144|
145| if len(k.J_Token) == 0 {
146| k.集_错误信息 = "获取到Token错误"
147| return false
148| }
149| return true
150|
151|}
152|
153|func (k *Api快验_类) Q取用户IP() string {
154| 请求json := make(map[string]interface{}, 5)
155| 请求json["Api"] = "GetUserIP"
156|
157| 响应json, ok := k.通讯(请求json)
158| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
159| return ""
160| }
161|
162| // {"Data":{"IP":"192.168.1.1"},"Time":1683005833,"Status":41177,"Note":""}
163| return string(响应json.GetStringBytes("Data", "IP"))
164|
165|}
166|
167|/*
168|.子程序 D登录_通用, 逻辑型, 公开, 成功返回真
169|.参数 响应信息json, 文本型, 参考, 登录后返回的用户信息 key绑定信息,OutUser顶掉其他同账号在线数量,VipTime,到期时间戳
    {"Key":"677F23CB3FA0055B5FD03916D6AB3C9A","OutUser":1,"VipTime":1685941943}
170|.参数 账号或卡号, 文本型, , 登录账号
171|.参数 密码, 文本型, 可空, 登陆密码 卡号模式空即可
172|.参数 绑定信息, 文本型, , 绑定信息,验证绑定,如果服务器绑定为空, 绑定该值
173|.参数 动态标记, 文本型, , 会显示在在线列表动态标记内,可以显示用户简单信息,可随时修改
174|.参数 当前版本, 文本型, , 当前软件版本 会判断是否为可用版本
175|*/
176|func (k *Api快验_类) D登录_通用(响应信息 *string, 账号或卡号, 密码, 绑定信息, 动态标记, 当前版本 string) bool {
177| 请求json := make(map[string]interface{}, 10)
178| 请求json["Api"] = "UserLogin"
179| 请求json["UserOrKa"] = 账号或卡号
180| 请求json["PassWord"] = 密码
181| 请求json["Key"] = 绑定信息
182| 请求json["Tab"] = 动态标记

```

```

183| 请求json["AppVer"] = 当前版本
184|
185| 响应json, ok := k.通讯(请求json)
186| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
187| return false
188| }
189|
190| // {"Data":{"Key":"677F23CB3FA0055B5FD03916D6AB3C9A","OutUser":1,"VipTime":1685941943},"Time":1683379761,"Status":36354,"Note":""}
191| *响应信息 = 响应json.GetObject("Data").String()
192|
193| return true
194|
195|}
196|
197|/*
198|子程序 Y用户减少余额, 逻辑型, 公开, 成功返回真 余额所有这个账号登录的应用都可以使用
199|参数 响应信息json, 文本型, 参考, {"Money":2382.56} 剩余金额
200|参数 减少数值, 双精度小数型, , 负数无效
201|参数 减少原因, 文本型, , 会写到日志记录
202|参数 一级代理用户ID, 整数型, 可空, 减少成功可以分成一定数量给代理 仅限一级代理用户id
因为二三级代理,余额完全由一级代理设置,独立全局系统金额体系外
203|参数 一级代理分成, 双精度小数型, 可空, 分成金额,不能超过,减少数值,
204|参数 分成原因, 文本型, 可空, 会写到余额日志记录
205|*/
206|
207|func (k *Api快验_类) Y用户减少余额(响应信息 *string, 减少数值 float64, 减少原因 string, 一级代理用户ID int, 一级代理分成
float64, 分成原因 string) bool {
208| 请求json := make(map[string]interface{}, 10)
209| 请求json["Api"] = "UserReduceMoney"
210| 请求json["Money"] = 减少数值
211| 请求json["Log"] = 减少原因
212| 请求json["AgentId"] = 一级代理用户ID
213| 请求json["AgentMoney"] = 一级代理分成
214| 请求json["AgentMoneyLog"] = 分成原因
215|
216| 响应json, ok := k.通讯(请求json)
217| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
218| return false
219| }
220|
221| // {"Data":{"Money":"9.93"},"Time":1683379761,"Status":36354,"Note":""}
222| *响应信息 = 响应json.GetObject("Data").String()
223|
224| return true
225|
226|}
227|
228|/*
229|.
230|子程序 用户减少积分, 逻辑型, 公开, 成功返回真
积分类似余额但是只有所属应用可以使用,建议和余额1:1兑换,只想本应用使用时操作 解决计时模式时
不想要用余额又没有变量控制按次收费的问题
231|参数 响应信息json, 文本型, 参考, {"VipNumber":"9.93"} 剩余积分
232|参数 减少数值, 双精度小数型, , 负数无效
233|参数 减少原因, 文本型, , 会写到日志记录
234|*/
235|func (k *Api快验_类) Y用户减少积分(响应信息 *string, 减少数值 float64, 减少原因 string) bool {
236| 请求json := make(map[string]interface{}, 10)
237| 请求json["Api"] = "UserReduceVipNumber"
238| 请求json["VipNumber"] = 减少数值

```

```

239| 请求json["Log"] = 减少原因
240|
241| 响应json, ok := k.通讯(请求json)
242| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
243| return false
244| }
245|
246| // {"Data":{"VipNumber":"9.93","Time":1683379761,"Status":36354,"Note":""}
247| *响应信息 = 响应json.GetObject("Data").String()
248|
249| return true
250|
251|}
252|
253|/*
254|.子程序 用户减少点数, 逻辑型, 公开, 成功返回真 只有计点方式才可以
255|.参数 响应信息json, 文本型, 参考, {"VipTime":"9"} 剩余点数
256|.参数 减少数值, 整数型, , 负数无效 只能为整数
257|.参数 减少原因, 文本型, , 会写到日志记录
258|*/
259|func (k *Api快验_类) Y用户减少点数(响应信息 *string, 减少数值 int, 减少原因 string) bool {
260| 请求json := make(map[string]interface{}, 10)
261| 请求json["Api"] = "UserReduceVipTime"
262| 请求json["VipTime"] = 减少数值
263| 请求json["Log"] = 减少原因
264|
265| 响应json, ok := k.通讯(请求json)
266| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
267| return false
268| }
269|
270| // {"Data":{"VipTime":"9","Time":1683379761,"Status":36354,"Note":""}
271| *响应信息 = 响应json.GetObject("Data").String()
272| return true
273|}
274|
275|/*
276|.子程序 Q取服务器连接状态, 逻辑型, 公开, 成功返回真 异常返回假
277|*/
278|func (k *Api快验_类) Q取服务器连接状态() bool {
279| 请求json := make(map[string]interface{}, 5)
280| 请求json["Api"] = "IsServerLink"
281|
282| _, ok := k.通讯(请求json)
283| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
284| return false
285| }
286|
287| return true
288|}
289|
290|/*
291|.子程序 取登录状态, 逻辑型, 公开, 登录状态正常返回真 异常返回假比如未登录或心跳过期,注意vip过期不会注销登录状态
292|*/
293|func (k *Api快验_类) Q取登录状态() bool {
294| 请求json := make(map[string]interface{}, 5)
295| 请求json["Api"] = "IsLogin"
296|
297| _, ok := k.通讯(请求json)
298| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了

```

```

299| return false
300| }
301|
302| return true
303|}
304|
305|/*
306|子程序 取Vip数据, 逻辑型, 公开, 正常返回真 异常返回假
307|*/
308|func (k *Api快验_类) Q取Vip数据(响应信息 *string) bool {
309|  请求json := make(map[string]interface{}, 5)
310|  请求json["Api"] = "GetVipData"
311|
312|  响应json, ok := k.通讯(请求json)
313|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
314|    return false
315|  }
316|  // {"Time":1683379761,"Status":203,"Note":"Vip已到期"}
317|  // {"Time":1683379761,"Status":208,"Note":"Vip数据非标准Json"}
318|  // {"Data":{"VipData":"这里的数据,只有登录成功并且账号会员不过期才会传输出去的数据","VipData2":"这里的数据,只有登录成功并且账号会员不过期才会传输出去的数据"},"Time":1683463084,"Status":16986,"Note":""}
319|  *响应信息 = 响应json.GetObject("Data").String()
320|  return *响应信息 != ""
321|}
322|
323|/*
324|子程序 取应用公告, 文本型, 公开, 正常返回公告 异常返回空
325|参数 响应信息, 文本型, 参考 可空
326|*/
327|func (k *Api快验_类) Q取应用公告(响应信息 *string) string {
328|  请求json := make(map[string]interface{}, 5)
329|  请求json["Api"] = "GetAppGongGao"
330|
331|  响应json, ok := k.通讯(请求json)
332|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
333|    return ""
334|  }
335|  *响应信息 = 响应json.GetObject("Data").String()
336|  return string(响应json.GetStringBytes("AppGongGao"))
337|}
338|
339|/*
340|子程序 取应用专属变量, 逻辑型, 公开, 正常返回真 异常返回假
341|参数 响应信息, 文本型, 参考 可空, 传入变量 赋值变量文本值 如果是逻辑值 1=真 0=假
342|参数 变量名称, 文本型, , 变量名称
343|*/
344|func (k *Api快验_类) Q取应用专属变量(响应信息 *string, 变量名称 string) bool {
345|  请求json := make(map[string]interface{}, 5)
346|  请求json["Api"] = "GetAppPublicData"
347|  请求json["Name"] = 变量名称
348|
349|  响应json, ok := k.通讯(请求json)
350|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
351|    return false
352|  }
353|  /*
354|  ' {"Data":{"紧急公告":"我是一条公告"},"Time":1683473651,"Status":16968,"Note":""}
355|  ' {"Time":1683473472,"Status":208,"Note":"变量不存在"}
356|  ' {"Time":1683473536,"Status":209,"Note":"未登录,请先操作登录"}
357|  */

```

```

358| *响应信息 = 响应json.GetObject("Data").String()
359| return true
360|}
361|
362|/*
363|.子程序 取公共变量, 逻辑型, 公开, 所有软件不用登录都可以读取的变量
364|.参数 响应信息, 文本型, 参考 可空, 传入变量 赋值变量文本值 如果是逻辑值 1=真 0=假
365|.参数 变量名称, 文本型, , 变量名称
366|*/
367|func (k *Api快验_类) Q取公共变量(响应信息 *string, 变量名称 string) bool {
368| 请求json := make(map[string]interface{}, 5)
369| 请求json["Api"] = "GetPublicData"
370| 请求json["Name"] = 变量名称
371|
372| 响应json, ok := k.通讯(请求json)
373| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
374| return false
375| }
376| *响应信息 = 响应json.GetObject("Data").String()
377| return true
378|}
379|
380|/*
381|.
382|.子程序 取最新版本检测, 逻辑型, 公开, 检查成功返回真,检查失败返回假
383|.参数 响应信息, 文本型, 参考 可空, 响应信息
384|.参数 当前版本号, 文本型, , 大版本号.小版本号.编译版本号
385|.参数 检测编译版本号, 逻辑型, 可空, 是否检测编译版本号,建议自动检测值为假,用户主动检测值为真
386|.参数 是否需要更新, 逻辑型, 参考 可空, 传入变量 值为真 需要更新,值为假不更新
387|.参数 最新版本号文本, 文本型, 参考 可空, 传入变量 文本型版本号,"大版本号.小版本号.编译版本号" 版本设置第一行 用来显示
388|*/
389|func (k *Api快验_类) Q取最新版本检测(响应信息 *string, 当前版本号 string, 检测编译版本号 bool, 是否需要更新 *bool,
390| 最新版本号文本 *string) bool {
391| 请求json := make(map[string]interface{}, 10)
392| 请求json["Api"] = "GetAppVersion"
393| 请求json["Name"] = 当前版本号
394| 请求json["IsVersionAll"] = 检测编译版本号
395|
396| 响应json, ok := k.通讯(请求json)
397| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
398| return false
399| }
400| // {"Data":{"IsUpdate":true,"NewVersion":"1.2.5","Version":1.2,"Time":1683542365,"Status":28677,"Note":""}}
401| *响应信息 = 响应json.GetObject("Data").String()
402| *是否需要更新 = 响应json.GetBool("Data", "IsUpdate")
403| *最新版本号文本 = string(响应json.GetStringBytes("Data", "NewVersion"))
404| return true
405|}
406|/*
407|.
408|.子程序 取新版本下载地址, 逻辑型, 公开, 所有软件不用登录都可以读
409|.参数 新版本下载地址, 文本型, 参考
410|*/
411|func (k *Api快验_类) Q取新版本下载地址(新版本下载地址 *string) bool {
412| 请求json := make(map[string]interface{}, 10)
413| 请求json["Api"] = "GetAppUpDataJson"
414|

```

```

415| 响应json, ok := k.通讯(请求json)
416| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
417| return false
418| }
419| *新版本下载地址 = string(响应json.GetStringBytes("Data", "AppUpDataJson"))
420| return true
421|}
422|
423|/*
424|.子程序 取应用主页Url, 逻辑型, 公开, 所有软件不用登录都可以读
425|.参数 主页Url, 文本型, 参考
426|*/
427|func (k *Api快验_类) Q取应用主页Url(主页Url *string) bool {
428| 请求json := make(map[string]interface{}, 10)
429| 请求json["Api"] = "GetAppHomeUrl"
430|
431| 响应json, ok := k.通讯(请求json)
432| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
433| return false
434| }
435| //{ "Data":{ "AppHomeUrl": "www.baidu.com", "Time":1683473651, "Status":16968, "Note":"" }
436| *主页Url = string(响应json.GetStringBytes("Data", "AppHomeUrl"))
437| return true
438|}
439|
440|/*
441|.子程序 置新绑定信息, 逻辑型, 公开, 换绑成功返回真,失败返回假
442|.参数 响应信息, 文本型, 参考 可空, 响应信息{"ReduceVipTime":10} 换绑扣除多少点数或时间
443|.参数 新绑定信息, 文本型
444|.参数 账号或卡号, 文本型, 可空, 账号或卡号 ,仅用在未登录时想更换绑定
445|.参数 密码, 文本型, 可空, 密码 如果是卡号 就空即可
446|*/
447|func (k *Api快验_类) Z置新绑定信息(响应信息 *string, 新绑定信息, 账号或卡号, 密码 string) bool {
448| 请求json := make(map[string]interface{}, 10)
449| 请求json["Api"] = "SetAppUserKey"
450| 请求json["NewKey"] = 新绑定信息
451| 请求json["User"] = 账号或卡号
452| 请求json["PassWord"] = 密码
453|
454| 响应json, ok := k.通讯(请求json)
455| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
456| return false
457| }
458| //{ "Data":{ "ReduceVipTime":10, "Time":1683601988, "Status":34623, "Note":"" }
459| //{ "Time":1683596452, "Status":210, "Note":"未登录,请先操作登录"}
460| //{ "Time":1683595169, "Status":205, "Note":"新绑定信息不能为空."}
461| *响应信息 = 响应json.GetObject("Data").String()
462| return true
463|}
464|
465|/*
466|.子程序 置新用户消息, 逻辑型, 公开, 成功返回真,失败返回假
467|.参数 消息类型, 整数型, , 1 其他, 2 bug提交 , 3 投诉建议
468|.参数 消息内容, 文本型
469|*/
470|func (k *Api快验_类) Z置新用户消息(消息类型 int, 消息内容 string) bool {
471| 请求json := make(map[string]interface{}, 10)
472| 请求json["Api"] = "SetNewUserMsg"
473| 请求json["MsgType"] = 消息类型
474| 请求json["Msg"] = 消息内容

```

```

475|
476| _ ok := k.通讯(请求json)
477| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
478| return false
479| }
480| return true
481|}
482|
483|/*
484|.子程序 取验证码, 逻辑型, 公开, 成功返回真,失败返回假
485|.参数 响应信息, 文本型, 参考, 解码后如果无法显示,删除data:image/png;base64, 在解码
486|.参数 类型, 整数型, , 验证码类型,默认 1 英数验证码,2滑块验证码
487|*/
488|func (k *Api快验_类) Q取验证码(响应信息 *string, 类型 int) bool {
489| 请求json := make(map[string]interface{}, 10)
490| 请求json["Api"] = "GetCaptcha"
491| 请求json["CaptchaType"] = 类型
492|
493| 响应json, ok := k.通讯(请求json)
494| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
495| return false
496| }
497|
498| *响应信息 = 响应json.GetObject("Data").String()
499|
500| return true
501|}
502|
503|/*
504|.子程序 取短信验证码, 逻辑型, 公开, 成功返回真,失败返回假
505|.参数 响应信息, 文本型, 参考, {"Captchalid":"4T7fSxvHV75tfgg","CaptchaType":3}
506|.参数 手机号, 文本型, 可空
507|.参数 用户名, 文本型, 可空, 如果手机号为空 可以填写用户名,会向用户名的手机号发送,找回密码时可以使用
508|*/
509|func (k *Api快验_类) Q取短信验证码(响应信息 *string, 手机号, 用户名 string) bool {
510| 请求json := make(map[string]interface{}, 10)
511| 请求json["Api"] = "GetSMSCaptcha"
512| 请求json["Phone"] = 手机号
513| 请求json["User"] = 用户名
514|
515| 响应json, ok := k.通讯(请求json)
516| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
517| return false
518| }
519|
520| *响应信息 = 响应json.GetObject("Data").String()
521| return true
522|}
523|
524|/*
525|.子程序 取绑定信息, 逻辑型, 公开, 登录状态正常返回真 异常返回假比如未登录或心跳过期
526|.参数 响应信息, 文本型, 参考, 绑定信息
527|*/
528|func (k *Api快验_类) Q取绑定信息(绑定信息 *string) bool {
529| 请求json := make(map[string]interface{}, 10)
530| 请求json["Api"] = "GetAppUserKey"
531|
532| 响应json, ok := k.通讯(请求json)
533| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
534| return false

```

```

535| }
536|
537| *绑定信息 = string(响应json.GetStringBytes("Data", "Key"))
538| return true
539|}
540|
541|/*
542|.子程序 取用户是否存在, 逻辑型, 公开, 正常返回真 异常返回假
543|.参数 是否存在, 逻辑型, 参考, 存在值为真,不存在值为假
544|.参数 用户名, 文本型, , 用户名称 或 卡号
545|*/
546|func (k *Api快验_类) Q取用户是否存在(是否存在 *bool, 用户名 string) bool {
547| 请求json := make(map[string]interface{}, 10)
548| 请求json["Api"] = "GetIsUser"
549| 请求json["User"] = 用户名
550|
551| 响应json, ok := k.通讯(请求json)
552| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
553| return false
554| }
555| /*
556| ' {"Data":{"IsUser":false},"Time":1683826090,"Status":34224,"Note":""}
557| ' {"Data":{"IsUser":ture},"Time":1683826090,"Status":34224,"Note":""}
558| */
559| *是否存在 = 响应json.GetBool("Data", "IsUser")
560| return true
561|}
562|
563|/*
564|.子程序 取软件用户信息, 逻辑型, 公开, 正常返回真 异常返回假 获取软件用户相关信息
565|.参数 响应信息, 文本型, 参考
566|*/
567|func (k *Api快验_类) Q取软件用户信息(响应信息 *string, AppVer string) bool {
568| 请求json := make(map[string]interface{}, 10)
569| 请求json["Api"] = "GetAppUserInfo"
570| 请求json["AppVer"] = AppVer
571|
572| 响应json, ok := k.通讯(请求json)
573| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
574| return false
575| }
576| /*' {"Id":1,"Key":"aaaaaa","MaxOnline":1,"LoginIp":"127.0.0.1","RegisterTime":1683349292,"LoginTime": 1683349292,"Sta
tus":1,"Uid":21,"User":"aaaaaa","UserClassId":22,"UserClassMark":2,"UserClassName":"Vip2","UserClassWeight":2,"VipNumbe
r":115.78,"VipTime":1715438220}
577| *响应信息 = 响应json.GetObject("Data").String()
578| return true
579|}
580|
581|/*
582|.子程序 取用户基础信息, 逻辑型, 公开, 正常返回真 异常返回假 获取 邮箱 手机号 QQ 是否已实名等基础信息
583|.参数 响应信息, 文本型, 参考, {"Email":"1056795985@qq.com","Phone":"15666666666","Qq":"1056795985"}
584|*/
585|func (k *Api快验_类) Q取用户基础信息(响应信息 *string) bool {
586| 请求json := make(map[string]interface{}, 10)
587| 请求json["Api"] = "GetUserInfo"
588|
589| 响应json, ok := k.通讯(请求json)
590| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
591| return false
592| }

```

```

593| // {"Email":"1056795985@qq.com","Phone":"15666666666","Qq":"1056795985"}
594| *响应信息 = 响应json.GetObject("Data").String()
595| return true
596|}
597|
598|/*
599|.子程序 置用户基础信息, 逻辑型, 公开, 正常返回真 异常返回假 设置 邮箱手机号 等基础信息
600|.参数 QQ, 文本型
601|.参数 邮箱, 文本型
602|.参数 手机号, 文本型
603|*/
604|func (k *Api快验_类) Z置用户基础信息(QQ, 邮箱, 手机号 string) bool {
605| 请求json := make(map[string]interface{}, 10)
606| 请求json["Api"] = "SetUserQqEmailPhone"
607| 请求json["Qq"] = QQ
608| 请求json["Email"] = 邮箱
609| 请求json["Phone"] = 手机号
610|
611| _, ok := k.通讯(请求json)
612| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
613| return false
614| }
615|
616| return true
617|}
618|
619|/*
620|.子程序 用户注册, 逻辑型, 公开, 正常返回真 异常返回假
621|.参数 注册账号, 文本型
622|.参数 密码, 文本型
623|.参数 绑定信息, 文本型
624|.参数 超级密码, 文本型
625|.参数 QQ, 文本型, 可空
626|.参数 邮箱, 文本型, 可空
627|.参数 手机号, 文本型, 可空
628|*/
629|func (k *Api快验_类) Y用户注册(注册账号, 密码, 绑定信息, 超级密码, Qq, 邮箱, 手机号 string) bool {
630| 请求json := make(map[string]interface{}, 10)
631| 请求json["Api"] = "NewUserInfo"
632| 请求json["User"] = 注册账号
633| 请求json["PassWord"] = 密码
634| 请求json["Key"] = 绑定信息
635| 请求json["SuperPassWord"] = 超级密码
636| 请求json["Qq"] = Qq
637| 请求json["Email"] = 邮箱
638| 请求json["Phone"] = 手机号
639|
640| _, ok := k.通讯(请求json)
641| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
642| return false
643| }
644| // {"Time":1684035704,"Status":11707,"Note":"注册成功"}
645| // {"Time":1684034845,"Status":200,"Note":"email邮箱格式不正确"}
646| // {"Time":1684034898,"Status":200,"Note":"超级密码以字母开头, 长度在6-18之间, 只能包含字符、数字和下划线"}
647| // {"Time":1684035056,"Status":200,"Note":"超级密码不能和密码相同"}
648| // {"Time":1684035081,"Status":200,"Note":"用户已存在"}
649| return true
650|}
651|
652|/*

```

```

653|.子程序 取系统时间戳, 整数型, 公开, 正常返回十位时间戳 异常返回0
654|.参数 响应信息, 文本型, 参考, 仅供参考 {"Time":1684036534}
655|*/
656|func (k *Api快验_类) Q取系统时间戳() int {
657|  请求json := make(map[string]interface{}, 5)
658|  请求json["Api"] = "GetSystemTime"
659|
660|  响应json, ok := k.通讯(请求json)
661|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
662|    return 0
663|  }
664|
665|  // {"Data":{"Time":1684036534},"Time":1684036534,"Status":17609,"Note":""}
666|  return 响应json.GetInt("Data", "Time")
667|
668|}
669|
670|/*
671|.子程序 取软件用户备注, 逻辑型, 公开
672|.参数 响应信息, 文本型, 参考 可空, 备注信息
673|*/
674|func (k *Api快验_类) Q取软件用户备注(响应信息 *string) bool {
675|  请求json := make(map[string]interface{}, 10)
676|  请求json["Api"] = "GetAppUserNote"
677|
678|  响应json, ok := k.通讯(请求json)
679|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
680|    return false
681|  }
682|
683|  *响应信息 = string(响应json.GetStringBytes("Data", "Note"))
684|  return true
685|}
686|
687|/*
688|.子程序 取vip到期时间戳, 整数型, 公开, 正常返回十位时间戳 异常返回0
689|*/
690|func (k *Api快验_类) Q取vip到期时间戳() int {
691|  var 时间戳 = 0
692|  k.Q取vip剩余点数(&时间戳)
693|  return 时间戳
694|}
695|
696|func (k *Api快验_类) Q取vip剩余点数(剩余点数 *int) bool {
697|  请求json := make(map[string]interface{}, 5)
698|  请求json["Api"] = "GetAppUserVipTime"
699|
700|  响应json, ok := k.通讯(请求json)
701|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
702|    return false
703|  }
704|  // {"Data":{"VipTime":1714919820},"Time":1684036726,"Status":15489,"Note":""}
705|  *剩余点数 = 响应json.GetInt("Data", "VipTime")
706|
707|  return true
708|}
709|
710|/*
711|.子程序 用户登录注销, 逻辑型, 公开, 成功返回真,失败返回假
712|*/

```

```

713|func (k *Api快验_类) Y用户登录注销() bool {
714|  请求json := make(map[string]interface{}, 10)
715|  请求json["Api"] = "LogOut"
716|
717|  _, ok := k.通讯(请求json)
718|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
719|  return false
720|  }
721|  k.J_Token = ""
722|  return true
723|}
724|
725|/*
726|.子程序 用户登录注销_远程, 逻辑型, 公开, 成功返回真,失败返回假 会注销全部本应用的在线账号
727|.参数 用户名或卡号, 文本型
728|.参数 密码, 文本型, , 无密码直接空即可
729|*/
730|func (k *Api快验_类) Y用户登录注销_远程(用户名或卡号, 密码 string) bool {
731|  请求json := make(map[string]interface{}, 10)
732|  请求json["Api"] = "RemoteLogOut"
733|  请求json["User"] = 用户名或卡号
734|  请求json["PassWord"] = 密码
735|
736|  _, ok := k.通讯(请求json)
737|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
738|  return false
739|  }
740|  return true
741|}
742|
743|/*
744|.子程序 心跳, 逻辑型, 公开, 成功返回真,失败返回假
745|.参数 响应信息, 文本型, 参考 可空, 仅供参考
746|.参数 响应当前状态, 整数型, 参考 可空, 当前状态 正常返回1 会员已到期返回3(免费模式即使到期了也不会返回3)
747|*/
748|func (k *Api快验_类) X心跳(响应信息 *string, 响应当前状态 *int) bool {
749|  请求json := make(map[string]interface{}, 10)
750|  请求json["Api"] = "HeartBeat"
751|
752|  响应json, ok := k.通讯(请求json)
753|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
754|  return false
755|  }
756|  // {"Time":1683601988,"Status":34623,"Note":""}
757|  // {"Time":1683596452,"Status":210,"Note":"未登录,请先操作登录"}
758|  // {"Data":{"Status":1},"Time":1684038983,"Status":35387,"Note":""}
759|  *响应信息 = 响应json.GetObject("Data").String()
760|  *响应当前状态 = 响应json.GetInt("Data", "Status")
761|  return true
762|}
763|
764|/*
765|.子程序 密码找回或修改_超级密码, 逻辑型, 公开, 成功返回真,失败返回假 修改成功后会注销所有在线的账号
766|.参数 响应信息, 文本型, 参考 可空, 仅供参考
767|.参数 用户名, 文本型
768|.参数 新密码, 文本型
769|.参数 超级密码, 文本型
770|*/
771|func (k *Api快验_类) M密码找回或修改_超级密码(响应信息 *string, 用户名, 新密码, 超级密码 string) bool {
772|  请求json := make(map[string]interface{}, 10)

```

```

773| 请求json["Api"] = "SetPassWord"
774| 请求json["Type"] = 1
775| 请求json["User"] = 用户名
776| 请求json["NewPassWord"] = 新密码
777| 请求json["SuperPassWord"] = 超级密码
778|
779| 响应json, ok := k.通讯(请求json)
780| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
781| return false
782| }
783| *响应信息 = 响应json.GetObject("Data").String()
784| return true
785|}
786|
787|/*
788|.
789|.子程序 密码找回或修改_绑定手机, 逻辑型, 公开, 成功返回真,失败返回假 修改成功后会注销所有在线的账号
790|.参数 响应信息, 文本型, 参考 可空, 仅供参考
791|.参数 用户名, 文本型
792|.参数 新密码, 文本型
793|.参数 短信验证码Id, 文本型, 通过取短信验证码方式获取
794|.参数 短信验证码, 文本型
795|*/
796|func (k *Api快验_类) M密码找回或修改_绑定手机(用户名, 新密码, 短信验证码Id, 短信验证码 string) bool {
797| 请求json := make(map[string]interface{}, 10)
798| 请求json["Api"] = "SetPassWord"
799| 请求json["Type"] = 2
800| 请求json["User"] = 用户名
801| 请求json["NewPassWord"] = 新密码
802| 请求json["PhoneCaptchalId"] = 短信验证码Id
803| 请求json["PhoneCaptchaValue"] = 短信验证码
804|
805| _, ok := k.通讯(请求json)
806| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
807| return false
808| }
809| return true
810|}
811|
812|/*
813|.子程序 取用户余额, 逻辑型, 公开, 正常返回真 异常返回假
814|.参数 余额, 双精度小数型, 参考
815|*/
816|func (k *Api快验_类) Q取用户余额(余额 *float64) bool {
817| 请求json := make(map[string]interface{}, 10)
818| 请求json["Api"] = "GetUserRmb"
819|
820| 响应json, ok := k.通讯(请求json)
821| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
822| return false
823| }
824| // {"Data":{"Rmb":2375.49},"Time":1684064350,"Status":28373,"Note":""}
825| *余额 = 响应json.GetFloat64("Data", "Rmb")
826| return true
827|}
828|
829|/*
830|.子程序 取用户积分, 逻辑型, 公开, 正常返回真 异常返回假
831|.参数 积分, 双精度小数型, 参考
832|*/

```

```

833|func (k *Api快验_类) Q取用户积分(积分 *float64) bool {
834| 请求json := make(map[string]interface{}, 10)
835| 请求json["Api"] = "GetAppUserVipNumber"
836|
837| 响应json, ok := k.通讯(请求json)
838| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
839| return false
840| }
841| /*{"Data":{"VipNumber":108.78,"Time":1684064462,"Status":23298,"Note":""}
842| *积分 = 响应json.GetFloat64("Data", "VipNumber")
843| return true
844|}
845|
846|/*
847|.子程序 取开启验证码接口列表, 逻辑型, 公开, 正常返回真 异常返回假
848|.参数 需要验证码的接口列表, 文本型, 参考, {"api接口名": 需要验证码类型} 英数验证码=1 行为验证码=2 短信验证码=3
849|/*{"UserLogin":1,"UserReduceMoney":3,"UserReduceVipNumber":3,"UserReduceVipTime":3}
850|func (k *Api快验_类) Q取开启验证码接口列表(响应信息 *string) bool {
851| 请求json := make(map[string]interface{}, 10)
852| 请求json["Api"] = "GetCaptchaApiList"
853|
854| 响应json, ok := k.通讯(请求json)
855| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
856| return false
857| }
858| *响应信息 = string(响应json.GetStringBytes("Data"))
859| return true
860|}
861|
862|/*
863|.子程序 卡号充值, 逻辑型, 公开, 成功返回真,失败返回假
864|.参数 充值账号, 文本型, , 充值用户账号
865|.参数 充值卡号, 文本型
866|.参数 推荐人, 文本型, 可空, 推荐人 卡号如果推荐人有赠送,也会推荐的
867|/*
868|func (k *Api快验_类) K卡号充值(充值账号, 充值卡号, 推荐人 string) bool {
869| 请求json := make(map[string]interface{}, 10)
870| 请求json["Api"] = "UseKa"
871| 请求json["User"] = 充值账号
872| 请求json["Ka"] = 充值卡号
873| 请求json["InviteUser"] = 推荐人
874|
875| _, ok := k.通讯(请求json)
876| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
877| return false
878| }
879| /* {"Time":1683601988,"Status":34623,"Note":""}
880| ' {"Time":1684072661,"Status":200,"Note":"卡号已经使用到最大次数"}
881| ' {"Time":1684072730,"Status":200,"Note":"不是本应用卡号"}
882| ' {"Time":1684072730,"Status":200,"Note":"已使用本卡号充值过了,请勿重复充值"}
883| ' {"Time":1684072730,"Status":200,"Note":"用户已冻结,无法充值"}
884| ' {"Time":1684072730,"Status":200,"Note":"未注册应用,请先操作登录一次"}
885| ' {"Time":1684072730,"Status":200,"Note":"用户类型不同无法充值."}
886| */
887| return true
888|}
889|
890|/*
891|.子程序 取动态标记, 逻辑型, 公开,

```

```

892|.参数 动态标记, 文本型, 参考
893|*/
894|func (k *Api快验_类) Q取动态标记(动态标记 *string) bool {
895|  请求json := make(map[string]interface{}, 10)
896|  请求json["Api"] = "GetTab"
897|
898|  响应json, ok := k.通讯(请求json)
899|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
900|  return false
901|  }
902|  // {"Data":{"Tab":"test测试中英文"},"Time":1684076743,"Status":32888,"Note":""}
903|  *动态标记 = string(响应json.GetStringBytes("Data", "Tab"))
904|  return true
905|}
906|
907|/*
908|.子程序 置动态标记, 逻辑型, 公开,
909|.参数 动态标记, 文本型,
910|*/
911|func (k *Api快验_类) Z置动态标记(动态标记 string) bool {
912|  请求json := make(map[string]interface{}, 10)
913|  请求json["Api"] = "SetTab"
914|  请求json["Tab"] = 动态标记
915|
916|  _, ok := k.通讯(请求json)
917|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
918|  return false
919|  }
920|  return true
921|}
922|func (k *Api快验_类) D订单_取状态(响应信息 *string, 订单id string) bool {
923|  请求json := make(map[string]interface{}, 10)
924|  请求json["Api"] = "GetPayOrderStatus"
925|  请求json["OrderId"] = 订单id
926|
927|  响应json, ok := k.通讯(请求json)
928|  if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
929|  return false
930|  }
931|  *响应信息 = 响应json.GetObject("Data").String()
932|  return true
933|}
934|
935|/*
936|.
937|.子程序 余额充值_支付宝PC支付, 逻辑型, 公开,
938|.参数 响应信息, 文本型, 参考 可空, {"Status":3}
939|.参数 充值账号, 文本型, , 充值用户账号
940|.参数 充值金额, 双精度小数型, , 充值金额
941|.参数 支付Url, 文本型, 参考
942|.参数 订单id, 文本型, 参考 可空, 订单id 不为空时 为查询支付结果 响应信息Status= 1 未支付 2已支付未充值 3充值成功
4退款成功
943|*/
944|func (k *Api快验_类) Y余额充值_支付宝PC支付(响应信息 *string, 充值账号 string, 充值金额 float64, 支付Url, 订单id *string)
bool {
945|  请求json := make(map[string]interface{}, 10)
946|  请求json["Api"] = "GetAliPayPC"
947|  请求json["User"] = 充值账号
948|  请求json["Money"] = 充值金额
949|  请求json["OrderId"] = 订单id

```

```

950|
951| 响应json, ok := k.通讯(请求json)
952| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
953| return false
954| }
955| *响应信息 = 响应json.GetObject("Data").String()
956| *订单id = string(响应json.GetStringBytes("Data", "OrderId"))
957| *支付Url = string(响应json.GetStringBytes("Data", "PayURL"))
958| return true
959|}
960|
961|/*
962|.版本 2
963|
964|.子程序 订单_购买余额, 逻辑型, 公开, 获取支付地址,直接支付后充值到用户账号
965|.参数 响应信息, 文本型, 参考 可空, 根据支付通道的不同,会响应不同平台的信息
966|.参数 充值账号或卡号, 文本型, , 充值用户账号
967|.参数 充值金额, 双精度小数型, , 充值金额
968|.参数 支付通道, 文本型, 参考, "支付宝PC" "微信PC" "小叮当" 更多请查看系统管理->系统设置->在线支付设置
969|*/
970|func (k *Api快验_类) D订单_购买余额(响应信息 *string, 支付通道, 充值账号 string, 充值金额 float64, 订单id *string) bool {
971| 请求json := make(map[string]interface{}, 10)
972| 请求json["Api"] = "PayUserMoney"
973| 请求json["User"] = 充值账号
974| 请求json["Money"] = 充值金额
975| 请求json["PayType"] = 支付通道
976|
977| 响应json, ok := k.通讯(请求json)
978| if !ok { // 直接返回即可,错误原因 在 发包并返回解密 已经有了
979| return false
980| }
981| *响应信息 = 响应json.GetObject("Data").String()
982| *订单id = string(响应json.GetStringBytes("Data", "OrderId"))
983| return true
984|}
985|
986|/*
987|.子程序 余额充值_微信支付支付, 逻辑型, 公开,
988|.参数 响应信息, 文本型, 参考 可空, {"Status":3}
989|.参数 充值账号, 文本型, , 充值用户账号
990|.参数 充值金额, 双精度小数型, , 充值金额
991|.参数 支付二维码, 文本型, 参考, 文本生成二维码然后扫描就可以了 weixin://wxpay/bizpayurl?pr=QDKS4KWzz
992|.参数 订单id, 文本型, 参考 可空, 订单Id 不为空时 为查询支付结果 Status= 1 未支付 2已支付未充值 3充值成功 4退款成功
993|*/
994|func (k *Api快验_类) Y余额充值_微信支付支付(响应信息 *string, 充值账号 string, 充值金额 float64, 支付二维码, 订单id *string)
bool {
995| 请求json := make(map[string]interface{}, 10)
996| 请求json["Api"] = "GetWXPayPC"
27| 旧数据库, err := gorm.Open(mysql.New(mysql.Config))
28| if err != nil {
29| //链接失败了
30| fmt.Println("就数据库连接失败" + err.Error())
31| return nil
32| }
33| return 旧数据库 //返回连接好的db池
34| //获取gorm db对象, 其他包需要执行数据库查询的时候, 只要通过 global.GVA_DB 获取db对象即可。
35| //不用担心协程并发使用同样的db对象会共用同一个连接, db对象在调用他的方法的时候会从数据库连接池中获取新的连接
36|
37|}
38|func 连接旧数据库() *gorm.DB {

```

```

39|
40| mysqlConfig := mysql.Config{
41| DSN: "root:root@tcp(127.0.0.1:3306)/bsphp?charset=utf8mb4&parseTime=True&loc=Local", // DSN data source name
42| DefaultStringSize: 191, // string 类型字段的默认长度
43| SkipInitializeWithVersion: false, // 根据版本自动配置
44| }
45|
46| 旧数据库, err := gorm.Open(mysql.New(mysqlConfig))
47| if err != nil {
48| //链接失败了
49| fmt.Println("新数据库连接失败" + err.Error())
50| return nil
51| }
52| return 旧数据库 //返回连接好的db池
53| //获取gorm db对象, 其他包需要执行数据库查询的时候, 只要通过 global.GVA_DB 获取db对象即可。
54| //不用担心协程并发使用同样的db对象会共用同一个连接, db对象在调用他的方法的时候会从数据库连接池中获取新的连接
55|
56|}
57|func 插入测试用户() {
58| 旧数据库 := 连接旧数据库()
59| var 局_bsphp_user []bsphp_user
60| var Ty_user DB_User
61| var 总数 int64
62| _ = 旧数据库.Table("bsphp_user").Count(&总数).Find(&局_bsphp_user).Error
63| 新数据库 := 连接新数据库()
64| var err error
65| for _, 值 := range 局_bsphp_user {
66| Ty_user.Id = 0
67| Ty_user.User = 值.UserUser
68| Ty_user.PassWord = 值.UserPwd
69| Ty_user.Phone = 值.User_Mobile
70| Ty_user.Email = 值.UserEmail
71| Ty_user.Qq = 值.UserQq
72| Ty_user.SuperPassWord = 值.UserMibaoDaan
73| Ty_user.Status = int(值.User_IsLock)
74| Ty_user.Rmb = 值.UserRmb
75| Ty_user.RealNameAttestation = ""
76| //Ty_user.Role = int(值.UserDaili)
77| Ty_user.UPAgentId, _ = strconv.Atoi(值.UserAngetCarid)
78| Ty_user.AgentDiscount = int(值.User_Zhe)
79|
80| Ty_user.LoginIp = 值.User_LoginIp
81| if Ty_user.LoginIp == "" {
82| Ty_user.LoginAppid = 10000 + rand.Intn(5)
83| } else {
84| Ty_user.LoginAppid = 0
85| }
86| Ty_user.LoginTime = 值.User_LoginDate.Unix()
87| Ty_user.RegisterIp = 值.UserRelp
88| Ty_user.RegisterTime = 值.UserReDate.Unix()
89|
90| err = 新数据库.Table("db_User").Create(&Ty_user).Error
91| if err != nil {
92| fmt.Println("插入错误" + err.Error())
93| } else {
94| //
95| }
96| }
97| fmt.Printf("插用户成功%d", 总数)
98| if 旧数据库 != nil || 新数据库 != nil {

```

```

99| fmt.Println("aaa")
100| }
101|}
102|
103|type bsphp_user struct {
104| UserId int
105| UserName string
106| UserPwd string
107| User_IsLock int8
108| UserEmail string
109| UserMailOk int8
110| UserQq string
111| User_Mobile string
112| UserMibaoWenti string
113| UserMibaoDaan string
114| UserRmb float64
115| UserJifen int
116| UserRelp string
117| UserReDate time.Time
118| User_LoginIp string
119| User_LoginDate time.Time
120| User_CaoShi int
121| UserYao_User string
122| UserYao_Shu int
123| User_Zhe float64
124| User_LoginNum int
125| User_DenJiTmp int
126| User_DenJi int
127| UserDaili int8
128| UserIsPwd int `comment:"是否禁止修改密码"`
129| UserBeizhu string `comment:"用户备注"`
130| UserAngetCarid string `comment:"代理卡串"`
131|}
132|
133|type BsphpLog struct {
134| Id int
135| Leixing string
136| Date int
137| Ip string
138| Test string
139| User string
140|}
141|
142|func 插入用户登录日志() {
143| 旧数据库 := 连接旧数据库()
144| var 局_BsphpLog []BsphpLog
145| var Ty_user DB_LogLogin
146| var 总数 int64
147| _ = 旧数据库.Table("bsphp_log").Where("leixing=?", "user_login_log").Count(&总数).Find(&局_BsphpLog).Error
148| 新数据库 := 连接新数据库()
149| 角色 := 0
150| var err error
151| for _, 值 := range 局_BsphpLog {
152| Ty_user.Id = 0
153| Ty_user.User = 值.User
154| Ty_user.Time = int64(值.Date)
155| Ty_user.Ip = 值.Ip
156| if 值.User == "admin" {
157| Ty_user.Note = "管理员登录"
158| } else if 值.User == "ssssss" || 值.User == "17835695599" || 值.User == "452068926" {

```

```

159| Ty_user.Note = "一级代理登录"
160| } else {
161| Ty_user.Note = "用户登录"
162| }
163|
164| 角色 = 0
165| err = 新数据库.Model(局_BsphpLog).Table("db_User").Select("Role").Where("User = ?", 值.User).First(&角色).Error
166| if err != nil {
167| fmt.Println("查询错误" + err.Error())
168| }
169| Ty_user.LoginType = 角色
170|
171| err = 新数据库.Table("db_log_login").Create(&Ty_user).Error
172| if err != nil {
173| fmt.Println("插入错误" + err.Error())
174| } else {
175| //
176| }
177| }
178| fmt.Printf("插用户成功%d", 总数)
179| if 旧数据库 != nil || 新数据库 != nil {
180| fmt.Println("aaa")
181| }
182|}
183|
184|/*func 插入余额变化日志() {
185| 旧数据库 := 连接旧数据库()
186| var 局_BsphpLog []BsphpLog
187| var Ty_user DB_LogMoney
188| var 总数 int64
189| _ = 旧数据库.Table("bsphp_log").Where("leixing=?", "money_buy_log").Count(&总数).Find(&局_BsphpLog).Error
190| 新数据库 := 连接新数据库()
191| var err error
192| for _, 值 := range 局_BsphpLog {
193| Ty_user.Id = 0
194| Ty_user.User = 值.User
195| Ty_user.Time = 值.Date
196| Ty_user.Ip = 值.Ip
197| Ty_user.Count, _ = strconv.ParseFloat(W文本_取出中间文本(值.Test+"###", "|", "###"), 64)
198| Ty_user.Count = -Ty_user.Count
199| Ty_user.Note = 值.Test
200| err = 新数据库.Table("db_log_money").Create(&Ty_user).Error
201|
202| if err != nil {
203| fmt.Println("插入错误" + err.Error())
204| } else {
205| //
206| }
207| }
208| fmt.Printf("插用户成功" + strconv.FormatInt(总数, 10))
209|}*/
210|
211|type BsphpRmbPayLog struct {
212| Id int
213| PayId string
214| PayUid int
215| PayLei string
216| PayRbm float64
217| PayZhuangtai int8
218| PayInfo1 string

```

```

219| PayInfo2 string
220| PayDate time.Time
221|}
222|
223|func 插入余额充值日志() {
224| 旧数据库 := 连接旧数据库()
225| var 局_BsphpRmbPayLog []BsphpRmbPayLog
226| var Ty_user DB_LogRMBPayOrder
227| var 总数 int64
228| _ = 旧数据库.Table("bsphp_rmb_pay_log").Count(&总数).Find(&局_BsphpRmbPayLog).Error
229| 新数据库 := 连接新数据库()
230| var err error
231| for _, 值 := range 局_BsphpRmbPayLog {
232|   Ty_user.Id = 0
233|   Ty_user.PayOrder = 值.PayId
234|   Ty_user.Uid = 值.PayUid
235|   Ty_user.Status = int(值.PayZhuangtai + 1)
236|   Ty_user.Type = 值.PayLei
237|   Ty_user.Rmb = 值.PayRbm
238|   Ty_user.Time = int64(int(值.PayDate.Unix()))
239|   Ty_user.Ip = strconv.Itoa(rand.Intn(255)) + "." + strconv.Itoa(rand.Intn(255)) + "." + strconv.Itoa(rand.Intn(255)) + "." +
   strconv.Itoa(rand.Intn(255))
240|   Ty_user.Note = "测试数据自动插入"
241|
242|   if Ty_user.Type != "zfbdiannao" {
243|     Ty_user.Note = Ty_user.Type
244|     Ty_user.Type = "zfbdiannao"
245|   }
246|
247|   err = 新数据库.Table("db_log_rmbpayorder").Create(&Ty_user).Error
248|
249|   if err != nil {
250|     fmt.Println("插入错误" + err.Error())
251|   } else {
252|     //
253|   }
254| }
255| fmt.Printf("插用户成功" + strconv.FormatInt(总数, 10))
256|}
===== 文件: structs\system.go =====
1|// 各种结构体 比如表结构体 请求 响应 结构体等等
2|package DB
===== 文件: utils\Aes.go =====
1|package utils
2|
3|import (
4| "bytes"
5| "crypto/aes"
6| "crypto/cipher"
7| "fmt"
8|)
9|
10|// 加密
11|func Aes加密_cbc192(orig string, key string) []byte {
12| // 转成字节数组
13| if len(key) != 24 {
14|   return []byte{}
15| }
16| origData := []byte(orig)
17| k := []byte(key)[:24]

```

```

18| // 分组密钥
19| // NewCipher该函数限制了输入k的长度必须为16, 24或者32
20| block, _ := aes.NewCipher(k)
21| // 获取密钥块的长度
22| blockSize := block.BlockSize()
23| // 补 全码
24| origData = PKCS7Padding(origData, blockSize)
25| // 加密模式 iv
26| iv := []byte{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
27| blockMode := cipher.NewCBCEncrypter(block, iv)
28|
29| // 创建数组
30| cryptd := make([]byte, len(origData))
31| // 加密
32| blockMode.CryptBlocks(cryptd, origData)
33|
34| return cryptd
35|}
36|
37|// 加密
38|func Aes加密_cbc192_2(加密数据 []byte, key string) []byte {
39| // 转成字节数组
40| if len(key) != 24 {
41| return []byte{}
42| }
43| origData := 加密数据
44| k := []byte(key)[:24]
45| // 分组密钥
46| // NewCipher该函数限制了输入k的长度必须为16, 24或者32
47| block, _ := aes.NewCipher(k)
48| // 获取密钥块的长度
49| blockSize := block.BlockSize()
50| // 补 全码
51| origData = PKCS7Padding(origData, blockSize)
52| // 加密模式 iv
53| iv := []byte{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
54| blockMode := cipher.NewCBCEncrypter(block, iv)
55|
56| // 创建数组
57| cryptd := make([]byte, len(origData))
58| // 加密
59| blockMode.CryptBlocks(cryptd, origData)
60|
61| return cryptd
62|}
63|func Aes加密_cbc192密匙字节数组(orig string, key []byte) []byte {
64| // 转成字节数组
65| if len(key) != 24 {
66| return []byte{}
67| }
68| origData := []byte(orig)
69| k := key
70| // 分组密钥
71| // NewCipher该函数限制了输入k的长度必须为16, 24或者32
72| block, _ := aes.NewCipher(k)
73| // 获取密钥块的长度
74| blockSize := block.BlockSize()
75| // 补 全码
76| origData = PKCS7Padding(origData, blockSize)
77| // 加密模式 iv

```

```

78| iv := []byte{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
79| blockMode := cipher.NewCBCEncrypter(block, iv)
80|
81| // 创建数组
82| cryted := make([]byte, len(origData))
83| // 加密
84| blockMode.CryptBlocks(cryted, origData)
85|
86| return cryted
87|}
88|func Aes解密_cbc192(加密数据 []byte, key string) string {
89| return Aes解密_cbc192字节集(加密数据, []byte(key))
90|}
91|func Aes解密_cbc192字节集(加密数据 []byte, key []byte) string {
92|
93| //mark 还会报panic len(src)%x.blockSize != 0 src=加密数据
没明白什么意思, //20250220感觉还是数据问题以后在解决//还是deepseek优秀,一下就找到原因了
94| if len(key) != 24 || len(加密数据) == 0 || len(加密数据)%aes.BlockSize != 0 {
95| return ""
96| }
97|
98| k := key[:24]
99| // 分组密钥
100| block, _ := aes.NewCipher(k)
101| // 获取密钥块的长度
102| //blockSize := block.BlockSize()
103| // 加密模式
104| iv := []byte{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
105| blockMode := cipher.NewCBCDecrypter(block, iv)
106| // 创建数组
107| orig := make([]byte, len(加密数据))
108| // 解密
109| blockMode.CryptBlocks(orig, 加密数据)
110| // 去补 全码
111| orig = PKCS7UnPadding(orig)
112| return string(orig)
113|}
114|func Aes解密_cbc192字节集2(加密数据 []byte, key []byte) []byte {
115| if len(key) != 24 || len(加密数据) < 16 {
116| return []byte{}
117| }
118| k := key[:24]
119| // 分组密钥
120| block, _ := aes.NewCipher(k)
121| // 获取密钥块的长度
122| //blockSize := block.BlockSize()
123| // 加密模式
124| iv := []byte{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
125| blockMode := cipher.NewCBCDecrypter(block, iv)
126| // 创建数组
127| orig := make([]byte, len(加密数据))
128| // 解密
129| blockMode.CryptBlocks(orig, 加密数据)
130| // 去补 全码
131| orig = PKCS7UnPadding(orig)
132| return orig
133|}
134|
135|// 补码

```

136|// AES加密数据块分组长度必须为128bit(byte[16]), 密钥长度可以是128bit(byte[16])、192bit(byte[24])、256bit(byte[32])中的任意一个。

```
137|func PKCS7Padding(ciphertext []byte, blocksize int) []byte {
138| padding := blocksize - len(ciphertext)%blocksize
139| padtext := bytes.Repeat([]byte{byte(padding)}, padding)
140| return append(ciphertext, padtext...)
141|}
```

```
142|
```

```
143|// 去码
```

```
144|func PKCS7UnPadding(origData []byte) []byte {
145|
```

```
146| length := len(origData)
```

```
147| if length-1 &lt; 0 {
```

```
148| return origData
```

```
149| }
```

```
150| unpadding := int(origData[length-1])
```

```
151| if length-unpadding &lt; 0 {
```

```
152| fmt.Printf("AESP7UnPadding 数据异常:%s", origData)
```

```
153| return origData
```

```
154| }
```

```
155| return origData[:length - unpadding]
```

```
156|}
```

```
===== 文件: utils\Ecc.go =====
```

```
1|package utils
```

```
2|
```

```
3|import (
```

```
4| "crypto/aes"
```

```
5| "crypto/cipher"
```

```
6| "crypto/ecdsa"
```

```
7| "crypto/elliptic"
```

```
8| "crypto/rand"
```

```
9| "crypto/sha256"
```

```
10| "crypto/x509"
```

```
11| "encoding/base64"
```

```
12| "errors"
```

```
13| "io"
```

```
14|)
```

```
15|
```

```
16|func Ecc_生成密钥对() (私钥Bsse64, 公钥Bsse64 string, err error) {
```

```
17|
```

```
18| // 1. 生成 ECC 密钥对
```

```
19| privateKey, err := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
```

```
20| if err != nil {
```

```
21| return
```

```
22| }
```

```
23|
```

```
24| // 2. 导出私钥为 Base64
```

```
25| 私钥Bsse64, err = exportPrivateKeyToBase64(privateKey)
```

```
26| if err != nil {
```

```
27| return
```

```
28| }
```

```
29| //fmt.Println("私钥 (Base64):", privateKeyBase64)
```

```
30|
```

```
31| // 3. 导出公钥为 Base64
```

```
32| 公钥Bsse64, err = exportPublicKeyToBase64(&privateKey.PublicKey)
```

```
33| if err != nil {
```

```
34| return
```

```
35| }
```

```
36| return
```

```
37|}
```

```

38|func Ecc_公钥加密(公钥Base64 string, 明文 []byte) (密文 []byte, err error) {
39|
40| var importedPublicKey *ecdsa.PublicKey
41| importedPublicKey, err = importPublicKeyFromBase64(公钥Base64) // 5. 从 Base64 导入公钥
42| if err != nil {
43| return
44| }
45| // 6. 使用公钥加密数据
46| 密文, err = encryptWithPublicKey(importedPublicKey, 明文)
47| return
48|}
49|
50|func Ecc_私钥解密(私钥Base64 string, 密文 []byte) (明文 []byte, err error) {
51|
52| var importedPrivateKey *ecdsa.PrivateKey
53| // 4. 从 Base64 导入私钥
54| importedPrivateKey, err = importPrivateKeyFromBase64(私钥Base64)
55| if err != nil {
56| return
57| }
58| // 6. 使用公钥加密数据
59| 明文, err = decryptWithPrivateKey(importedPrivateKey, 密文)
60| return
61|}
62|
63|// Ecc_私钥签名 使用私钥对数据进行签名
64|func Ecc_私钥签名(私钥Base64 string, 数据 []byte) (签名 []byte, err error) {
65| // 从 Base64 导入私钥
66| importedPrivateKey, err := importPrivateKeyFromBase64(私钥Base64)
67| if err != nil {
68| return nil, err
69| }
70|
71| // 计算数据的哈希值
72| hash := sha256.Sum256(数据)
73|
74| // 使用私钥对哈希值进行签名
75| 签名, err = ecdsa.SignASN1(rand.Reader, importedPrivateKey, hash[:])
76| if err != nil {
77| return nil, err
78| }
79|
80| return 签名, nil
81|}
82|
83|// Ecc_公钥验签 使用公钥验证签名
84|func Ecc_公钥验签(公钥Base64 string, 数据 []byte, 签名 []byte) (验证通过 bool, err error) {
85| // 从 Base64 导入公钥
86| importedPublicKey, err := importPublicKeyFromBase64(公钥Base64)
87| if err != nil {
88| return false, err
89| }
90|
91| // 计算数据的哈希值
92| hash := sha256.Sum256(数据)
93|
94| // 使用公钥验证签名
95| 验证通过 = ecdsa.VerifyASN1(importedPublicKey, hash[:], 签名)
96| return 验证通过, nil
97|}

```

```

98|
99|// 导出私钥为 Base64
100|func exportPrivateKeyToBase64(privateKey *ecdsa.PrivateKey) (string, error) {
101| privateKeyBytes, err := x509.MarshalECPrivateKey(privateKey)
102| if err != nil {
103| return "", err
104| }
105| return base64.StdEncoding.EncodeToString(privateKeyBytes), nil
106|}
107|
108|// 导出公钥为 Base64
109|func exportPublicKeyToBase64(publicKey *ecdsa.PublicKey) (string, error) {
110| publicKeyBytes, err := x509.MarshalPKIXPublicKey(publicKey)
111| if err != nil {
112| return "", err
113| }
114| return base64.StdEncoding.EncodeToString(publicKeyBytes), nil
115|}
116|
117|// 从 Base64 导入私钥
118|func importPrivateKeyFromBase64(privateKeyBase64 string) (*ecdsa.PrivateKey, error) {
119| privateKeyBytes, err := base64.StdEncoding.DecodeString(privateKeyBase64)
120| if err != nil {
121| return nil, err
122| }
123| return x509.ParseECPrivateKey(privateKeyBytes)
124|}
125|
126|// 从 Base64 导入公钥
127|func importPublicKeyFromBase64(publicKeyBase64 string) (*ecdsa.PublicKey, error) {
128| publicKeyBytes, err := base64.StdEncoding.DecodeString(publicKeyBase64)
129| if err != nil {
130| return nil, err
131| }
132| pub, err := x509.ParsePKIXPublicKey(publicKeyBytes)
133| if err != nil {
134| return nil, err
135| }
136| return pub.(*ecdsa.PublicKey), nil
137|}
138|
139|// 使用公钥加密数据
140|func encryptWithPublicKey(publicKey *ecdsa.PublicKey, data []byte) ([]byte, error) {
141| // 生成临时密钥对
142| tempPrivateKey, err := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
143| if err != nil {
144| return nil, err
145| }
146|
147| // 使用 ECDH 生成共享密钥
148| sharedSecretX, _ := publicKey.Curve.ScalarMult(publicKey.X, publicKey.Y, tempPrivateKey.D.Bytes())
149| sharedSecret := sha256.Sum256(sharedSecretX.Bytes())
150|
151| // 使用 AES 加密数据
152| block, err := aes.NewCipher(sharedSecret[:])
153| if err != nil {
154| return nil, err
155| }
156| ciphertext := make([]byte, aes.BlockSize+len(data))
157| iv := ciphertext[:aes.BlockSize]

```

```

158| if _, err := io.ReadFull(rand.Reader, iv); err != nil {
159| return nil, err
160| }
161| stream := cipher.NewCFBEncrypter(block, iv)
162| stream.XORKeyStream(ciphertext[aes.BlockSize:], data)
163|
164| // 返回临时公钥和加密数据
165| tempPublicKey := tempPrivateKey.PublicKey
166| tempPublicKeyBytes, err := x509.MarshalPKIXPublicKey(&tempPublicKey)
167| if err != nil {
168| return nil, err
169| }
170| return append(tempPublicKeyBytes, ciphertext...), nil
171|}
172|
173|// 使用私钥解密数据
174|func decryptWithPrivateKey(privateKey *ecdsa.PrivateKey, encryptedData []byte) ([]byte, error) {
175| // 解析临时公钥
176| tempPublicKeyBytes := encryptedData[:91] // P-256 公钥长度为 91 字节
177| tempPublicKey, err := x509.ParsePKIXPublicKey(tempPublicKeyBytes)
178| if err != nil {
179| return nil, err
180| }
181|
182| // 使用 ECDH 生成共享密钥
183| sharedSecretX, _ := tempPublicKey.(*ecdsa.PublicKey).Curve.ScalarMult(tempPublicKey.(*ecdsa.PublicKey).X,
tempPublicKey.(*ecdsa.PublicKey).Y, privateKey.D.Bytes())
184| sharedSecret := sha256.Sum256(sharedSecretX.Bytes())
185|
186| // 使用 AES 解密数据
187| block, err := aes.NewCipher(sharedSecret[:])
188| if err != nil {
189| return nil, err
190| }
191| ciphertext := encryptedData[91:]
192| if len(ciphertext) < aes.BlockSize {
193| return nil, errors.New("密文太短")
194| }
195| iv := ciphertext[:aes.BlockSize]
196| ciphertext = ciphertext[aes.BlockSize:]
197| stream := cipher.NewCFBDecrypter(block, iv)
198| stream.XORKeyStream(ciphertext, ciphertext)
199| return ciphertext, nil
200|}
==== 文件: utils\Map.go ====
1|package utils
2|
3|func Map取key整数数组(m map[int]string) []int {
4| // 数组默认长度为map长度,后面append时,不需要重新申请内存和拷贝,效率很高
5| j := 0
6| keys := make([]int, len(m))
7| for k := range m {
8| keys[j] = k
9| j++
10| }
11| return keys
12|}
13|
14|func Map取key文本数组(m map[string]string) []string {
15| // 数组默认长度为map长度,后面append时,不需要重新申请内存和拷贝,效率很高

```

```

16| j := 0
17| keys := make([]string, len(m))
18| for k := range m {
19| keys[j] = k
20| j++
21| }
22| return keys
23|}
===== 文件: utils\Qqwry\lp查省市.go =====
1| package Qqwry
2|
3| import (
4| _ "embed"
5|)
6|
7|// 查询次数较少,为了方便,还是每次打开查询关闭数据库,后去如果需要查询性能,在处理
8|func C查询IP归属地(ip地址 string) (string, error) {
9| //局_耗时 := time.Now().UnixMilli()
10| 省份, _ , err := QueryIP(ip地址)
11| if err != nil {
12| return "", err
13| }
14| //fmt.Printf("ip查询耗时:%dms,城市: %s, 运营商: %s\n", time.Now().UnixMilli()-局_耗时, city, isp)
15| return 省份, nil
16|}
===== 文件: utils\Qqwry\qqwry.go =====
1| package Qqwry
2|
3| import (
4| "EFunc/utils"
5| "bytes"
6| _ "embed"
7| "encoding/binary"
8| "errors"
9| "fmt"
10| "golang.org/x/text/encoding/simplifiedchinese"
11| "golang.org/x/text/transform"
12| "io"
13| "net"
14| "os"
15| "strings"
16| "sync"
17|)
18|
19|//go:embed qqwry.dat
20|var 中国Ip查询库 []byte
21|
22|// 自动加载数据库文件
23|func init() {
24| if utils.W文件_是否存在("qqwry.dat") {
25| err := LoadFile("qqwry.dat")
26| if err != nil {
27| LoadData(中国Ip查询库)
28| fmt.Printf("本地纯真IP数据库,已加载到内存失败已重新加载内置库\n")
29| } else {
30| fmt.Printf("本地纯真IP数据库,已加载到内存\n")
31| }
32|
33| } else {
34| LoadData(中国Ip查询库)

```

```

35| fmt.Printf("纯真IP数据库,已加载到内存\n")
36| }
37|
38|}
39|
40|var (
41| data []byte
42| dataLen uint32
43| ipCache = &sync.Map{}
44|)
45|
46|const (
47| indexLen = 7
48| redirectMode1 = 0x01
49| redirectMode2 = 0x02
50|)
51|
52|type cache struct {
53| City string
54| lsp string
55|}
56|
57|func byte3ToUInt32(data []byte) uint32 {
58| i := uint32(data[0]) & 0xff
59| i |= (uint32(data[1]) & 0xff) << 8
60| i |= (uint32(data[2]) & 0xff) << 16
61| return i
62|}
63|
64|func gb18030Decode(src []byte) string {
65| in := bytes.NewReader(src)
66| out := transform.NewReader(in, simplifiedchinese.GB18030.NewDecoder())
67| d, _ := io.ReadAll(out)
68| return string(d)
69|}
70|
71|func Ip查信息(要查询的IP string) (省市 string, 运行商 string, err error) {
72| return QueryIP(要查询的IP)
73|}
74|func Ip查信息2(要查询的IP string) string {
75| var 省份 = ""
76| var 运营商 = ""
77| 省份, 运营商, _ = QueryIP(要查询的IP)
78| return 省份 + " " + 运营商
79|}
80|
81|// QueryIP 从内存或缓存查询IP
82|func QueryIP(queryIp string) (city string, isp string, err error) {
83| if v, ok := ipCache.Load(queryIp); ok {
84| city = v.(cache).City
85| isp = v.(cache).lsp
86| return
87| }
88| ip := net.ParseIP(queryIp).To4()
89| if ip == nil {
90| err = errors.New("ip is not ipv4")
91| return
92| }
93| ip32 := binary.BigEndian.Uint32(ip)
94| posA := binary.LittleEndian.Uint32(data[:4])

```

```

95| posZ := binary.LittleEndian.Uint32(data[4:8])
96| var offset uint32 = 0
97| for {
98| mid := posA + (((posZ-posA)/indexLen)&gt;&gt;1)*indexLen
99| buf := data[mid : mid+indexLen]
100| _ip := binary.LittleEndian.Uint32(buf[:4])
101| if posZ-posA == indexLen {
102| offset = byte3ToUInt32(buf[4:])
103| buf = data[mid+indexLen : mid+indexLen+indexLen]
104| if ip32 &lt; binary.LittleEndian.Uint32(buf[:4]) {
105| break
106| } else {
107| offset = 0
108| break
109| }
110| }
111| if _ip &gt; ip32 {
112| posZ = mid
113| } else if _ip &lt; ip32 {
114| posA = mid
115| } else if _ip == ip32 {
116| offset = byte3ToUInt32(buf[4:])
117| break
118| }
119| }
120| if offset &lt;= 0 {
121| err = errors.New("ip not found")
122| return
123| }
124| posM := offset + 4
125| mode := data[posM]
126| var ispPos uint32
127| switch mode {
128| case redirectMode1:
129| posC := byte3ToUInt32(data[posM+1 : posM+4])
130| mode = data[posC]
131| posCA := posC
132| if mode == redirectMode2 {
133| posCA = byte3ToUInt32(data[posC+1 : posC+4])
134| posC += 4
135| }
136| for i := posCA; i &lt; dataLen; i++ {
137| if data[i] == 0 {
138| city = string(data[posCA:i])
139| break
140| }
141| }
142| if mode != redirectMode2 {
143| posC += uint32(len(city) + 1)
144| }
145| ispPos = posC
146| case redirectMode2:
147| posCA := byte3ToUInt32(data[posM+1 : posM+4])
148| for i := posCA; i &lt; dataLen; i++ {
149| if data[i] == 0 {
150| city = string(data[posCA:i])
151| break
152| }
153| }
154| ispPos = offset + 8

```

```

155| default:
156| posCA := offset + 4
157| for i := posCA; i < dataLen; i++ {
158| if data[i] == 0 {
159| city = string(data[posCA:i])
160| break
161| }
162| }
163| ispPos = offset + uint32(5+len(city))
164| }
165| if city != "" {
166| city = strings.TrimSpace(gb18030Decode([]byte(city)))
167| }
168| ispMode := data[ispPos]
169| if ispMode == redirectMode1 || ispMode == redirectMode2 {
170| ispPos = byte3ToUInt32(data[ispPos+1 : ispPos+4])
171| }
172| if ispPos > 0 {
173| for i := ispPos; i < dataLen; i++ {
174| if data[i] == 0 {
175| isp = string(data[ispPos:i])
176| if isp != "" {
177| if strings.Contains(isp, "CZ88.NET") {
178| isp = ""
179| } else {
180| isp = strings.TrimSpace(gb18030Decode([]byte(isp)))
181| }
182| }
183| break
184| }
185| }
186| }
187| ipCache.Store(queryIp, cache{City: city, Isp: isp})
188| return
189|}
190|
191|// LoadData 从内存加载IP数据库
192|func LoadData(database []byte) {
193| data = database
194| dataLen = uint32(len(data))
195|}
196|
197|// LoadFile 从文件加载IP数据库
198|func LoadFile(filepath string) (err error) {
199| data, err = os.ReadFile(filepath)
200| if err != nil {
201| return
202| }
203| dataLen = uint32(len(data))
204| return
205|}
===== 文件: utils\Rsa.go =====
1|package utils
2|
3|import (
4| "EFunc/utils"
5| "crypto"
6| "crypto/md5"
7| "crypto/rand"
8| "crypto/rsa"

```

```

9| "crypto/x509"
10| "encoding/base64"
11| "encoding/pem"
12| "errors"
13| "fmt"
14| "math/big"
15|)
16|
17|func Rsa私钥签名(明文 string, RSA私钥 string) string {
18|
19| pemKey := []byte(RSA私钥)
20|
21| data := []byte(明文)
22| hashMd5 := md5.Sum(data)
23| hashed := hashMd5[:]
24| block, _ := pem.Decode(pemKey)
25| if block == nil {
26| return ""
27| }
28| privateKey, err := x509.ParsePKCS1PrivateKey(block.Bytes) //大坑 必须校验是否 block是否为nil
否则block.Bytes卡主,类似进入许可区,
29| if err != nil {
30| return ""
31| }
32| // 感觉和私钥加密区别实际就是 hash 参数有区别, 如果没这个参数就是私钥加密明文,如果有这个参数就是私钥加密 hash(明文)
33| signature, err := rsa.SignPKCS1v15(rand.Reader, privateKey, crypto.MD5, hashed)
34| //return strings.ToUpper(hex.EncodeToString(signature)) hex过长,所以选择base64
35| //2688fef23ac239b50bef486bc5c53b700910a2bcfd9ab5159a19586fc77afcb35835f2b8630fcdabd95c89f3f4231c623bccf0
06d7cce97f57bafb4f5c3cb8cbdf38d59e608444eaffecf96bb30d987117c409ab50a668f3e637d19e488cb82edb39fc3bde50cd
d64494d2e8907b4cd91a8921e00ecb7c2ec36faf312db7589
36| //Joj+8jrCOBUl70hrxcU7cAkQorz9mrUVmhlYb8d6/LNYNfK4Yw/Nq9lciFP0lxxiO8zwBtfM6X9XuvtpXDy4y9841Z5ghETq/+
7Plrsw2YcRfECatQpmjz5jfRnkiMuC7bOfw73lDN1kSU0uiQe0zZGokh4A7LfC7Db68xLbdYk=
37| return utils.B编码_BASE64编码(signature)
38|
39|}
40|
41|// Rsa公钥验签
42|func Rsa公钥验签(明文 string, base64签名 string, RSA公钥 string) bool {
43| pemKey := []byte(RSA公钥)
44|
45| data := []byte(明文)
46| hashMd5 := md5.Sum(data)
47| hashed := hashMd5[:]
48|
49| // 解码签名
50| signature := utils.B编码_BASE64解码(base64签名)
51| if len(signature) == 0 {
52| return false
53| }
54|
55| block, _ := pem.Decode(pemKey)
56| if block == nil {
57| return false
58| }
59|
60| publicKeyInterface, err := x509.ParsePKIXPublicKey(block.Bytes)
61| if err != nil {
62| return false
63| }
64|

```

```

65| publicKey := publicKeyInterface.(*rsa.PublicKey)
66|
67| // 验签
68| err = rsa.VerifyPKCS1v15(publicKey, crypto.MD5, hashed, signature)
69| if err != nil {
70| return false
71| }
72|
73| return true
74|}
75|
76|// RSA公钥私钥产生
77|func GetRsaKey() (err error, PublicKey string, PrivateKey string) {
78| bits := 1024
79| privateKey, err := rsa.GenerateKey(rand.Reader, bits)
80| if err != nil {
81| return err, "", ""
82| }
83|
84| privateKeyStream := x509.MarshalPKCS1PrivateKey(privateKey)
85| //file, err := os.Create("./key/private.pem")
86| block := &pem.Block{
87| Type: "RSA PRIVATE KEY",
88| Bytes: privateKeyStream,
89| }
90| // _ = pem.Encode(file, block)
91| privateKeyStr := string(pem.EncodeToMemory(block))
92| publicKey := &privateKey.PublicKey
93| publicKeyStream, _ := x509.MarshalPKIXPublicKey(publicKey)
94| //publicKeyStream := x509.MarshalPKCS1PublicKey(publicKey)
95| //file, err = os.Create("./key/public.pem")
96| block = &pem.Block{
97| Type: "PUBLIC KEY",
98| Bytes: publicKeyStream,
99| }
100| // _ = pem.Encode(file, block)
101| publicKeyStr := string(pem.EncodeToMemory(block))
102| return err, publicKeyStr, privateKeyStr
103|}
104|
105|func Rsa私钥解密(Rsa私钥 []byte, 加密数据 []byte) string {
106| return string(Rsa私钥解密2(Rsa私钥, 加密数据))
107|}
108|func Rsa私钥解密2(Rsa私钥 []byte, 加密数据 []byte) []byte {
109|
110| if len(加密数据) == 0 || len(Rsa私钥) == 0 {
111| return []byte{}
112| }
113|
114| //2、pem decode,得到block的der编码数据
115| block, _ := pem.Decode(Rsa私钥)
116| if block == nil {
117| fmt.Printf("私钥载入失败可能格式不正确:%s\n", string(Rsa私钥))
118| return []byte{}
119| } //密钥错误
120| }
121| derText := block.Bytes //大坑 必须校验是否 block是否为nil 否则block.Bytes卡主,类似进入许可区,
122| //3、解码der得到私钥
123| privateKey, err := x509.ParsePKCS1PrivateKey(derText)
124| if err != nil {

```

```

125| return []byte{}
126| //密钥错误
127| }
128| //4、私钥解密
129| plainText, err := rsa.DecryptPKCS1v15(rand.Reader, privateKey, 加密数据)
130|
131| if err != nil {
132| return []byte{}
133| //解密失败
134| }
135| return plainText
136|
137|}
138|func Rsa公钥加密(公钥 string, 加密内容 []byte) (base64密文 string) {
139| //2、pem decode,得到block的der编码数据
140| block, _ := pem.Decode([]byte(公钥))
141| if block == nil {
142| fmt.Printf("Rsa公钥加密公钥载入失败可能格式不正确:%s\n", 公钥)
143| return ""
144| //密钥错误
145| }
146| derText := block.Bytes //大坑 必须校验是否 block是否为nil 否则block.Bytes卡主,类似进入许可区,
147| //3、解码der得到公钥
148| //publicKey, err := x509.ParsePKCS1PublicKey(derText) // 这个只能载入PKCS1的 但是目前公钥基本都是PKCS8的
149|
150| publicKeyInterface, err := x509.ParsePKIXPublicKey(derText) //这个只能载入PKCS8的公钥
151| if err != nil {
152| return ""
153| //密钥错误
154| }
155| //类型断言
156| publicKey := publicKeyInterface.(*rsa.PublicKey) //强制转换
157|
158| //4、公钥加密
159| cipherData, err := rsa.EncryptPKCS1v15(rand.Reader, publicKey, 加密内容)
160| if err != nil {
161| return
162| }
163| base64密文 = base64.StdEncoding.EncodeToString(cipherData)
164| return base64密文
165|
166|}
167|
168|// 参考地址 https://www.cnblogs.com/imlgc/p/7076313.html
169|func RSA私钥加密(Rsa私钥 []byte, 明文 []byte) (base64密文 string) {
170| if len(明文) == 0 || len(Rsa私钥) == 0 {
171| return ""
172| }
173|
174| //2、pem decode,得到block的der编码数据
175|
176| block, _ := pem.Decode(Rsa私钥)
177| if block == nil {
178| fmt.Printf("RSA私钥加密Rsa私钥载入失败可能格式不正确:%s\n", string(Rsa私钥))
179| return ""
180| //密钥错误
181| }
182| derText := block.Bytes //大坑 必须校验是否 block是否为nil 否则block.Bytes卡主,类似进入许可区,
183| //3、解码der得到私钥
184| privateKey, err := x509.ParsePKCS1PrivateKey(derText)

```

```

185|
186| signData, err := rsa.SignPKCS1v15(nil, privateKey, crypto.Hash(0), 明文)
187| if err != nil {
188| return ""
189| }
190| base64密文 = base64.StdEncoding.EncodeToString(signData)
191| return base64密文
192|}
193|func RSA公钥解密(公钥 string, 密文 []byte) (明文字节集 []byte) {
194|
195| //2、pem decode,得到block的der编码数据
196| block, _ := pem.Decode([]byte(公钥))
197| if block == nil {
198| fmt.Printf("RSA公钥解密Rsa公钥载入失败可能格式不正确:%s\n", 公钥)
199| return []byte{}
200| //密钥错误
201| }
202| derText := block.Bytes //大坑 必须校验是否 block是否为nil 否则block.Bytes卡主,类似进入许可区,
203|
204| //3、解码der得到公钥
205|
206| publicKey, err := x509.ParsePKCS1PublicKey(derText)
207| //3、解码der得到公钥
208| //publicKey, err := x509.ParsePKCS1PublicKey(derText) // 这个只能载入PKCS1的 但是目前公钥基本都是PKCS8的
209|
210| publicKeyInterface, err := x509.ParsePKIXPublicKey(derText) //这个只能载入PKCS8的公钥
211| if err != nil {
212| return 明文字节集
213| //密钥错误
214| }
215|
216| publicKey = publicKeyInterface.(*rsa.PublicKey) //强制转换
217|
218| 明文字节集, err = publicDecrypt(publicKey, crypto.Hash(0), nil, 密文)
219| if err != nil {
220| return 明文字节集
221| }
222| return 明文字节集
223|}
224|
225|// copy from crypt/rsa/pkcs1v5.go
226|var hashPrefixes = map[crypto.Hash][]byte{
227| crypto.MD5: {0x30, 0x20, 0x30, 0x0c, 0x06, 0x08, 0x2a, 0x86, 0x48, 0x86, 0xf7, 0x0d, 0x02, 0x05, 0x05, 0x00, 0x04, 0x10},
228| crypto.SHA1: {0x30, 0x21, 0x30, 0x09, 0x06, 0x05, 0x2b, 0x0e, 0x03, 0x02, 0x1a, 0x05, 0x00, 0x04, 0x14},
229| crypto.SHA224: {0x30, 0x2d, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x04, 0x05, 0x00, 0x04, 0x1c},
230| crypto.SHA256: {0x30, 0x31, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x01, 0x05, 0x00, 0x04, 0x20},
231| crypto.SHA384: {0x30, 0x41, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x02, 0x05, 0x00, 0x04, 0x30},
232| crypto.SHA512: {0x30, 0x51, 0x30, 0x0d, 0x06, 0x09, 0x60, 0x86, 0x48, 0x01, 0x65, 0x03, 0x04, 0x02, 0x03, 0x05, 0x00, 0x04, 0x40},
233| crypto.MD5SHA1: {}, // A special TLS case which doesn't use an ASN1 prefix.
234| crypto.RIPEMD160: {0x30, 0x20, 0x30, 0x08, 0x06, 0x06, 0x28, 0xcf, 0x06, 0x03, 0x00, 0x31, 0x04, 0x14},
235|}
236|
237|// copy from crypt/rsa/pkcs1v5.go
238|func encrypt(c *big.Int, pub *rsa.PublicKey, m *big.Int) *big.Int {
239| e := big.NewInt(int64(pub.E))
240| c.Exp(m, e, pub.N)

```

```

241| return c
242|}
243|
244|// copy from crypt/rsa/pkcs1v5.go
245|func pkcs1v15HashInfo(hash crypto.Hash, inLen int) (hashLen int, prefix []byte, err error) {
246| // Special case: crypto.Hash(0) is used to indicate that the data is
247| // signed directly.
248| if hash == 0 {
249| return inLen, nil, nil
250| }
251|
252| hashLen = hash.Size()
253| if inLen != hashLen {
254| return 0, nil, errors.New("crypto/rsa: input must be hashed message")
255| }
256| prefix, ok := hashPrefixes[hash]
257| if !ok {
258| return 0, nil, errors.New("crypto/rsa: unsupported hash function")
259| }
260| return
261|}
262|
263|// copy from crypt/rsa/pkcs1v5.go
264|func leftPad(input []byte, size int) (out []byte) {
265| n := len(input)
266| if n > size {
267| n = size
268| }
269| out = make([]byte, size)
270| copy(out[len(out)-n:], input)
271| return
272|}
273|func unLeftPad(input []byte) (out []byte) {
274| n := len(input)
275| t := 2
276| for i := 2; i < n; i++ {
277| if input[i] == 0xff {
278| t = t + 1
279| } else {
280| if input[i] == input[0] {
281| t = t + int(input[1])
282| }
283| break
284| }
285| }
286| out = make([]byte, n-t)
287| copy(out, input[t:])
288| return
289|}
290|
291|// copy&modified from crypt/rsa/pkcs1v5.go
292|func publicDecrypt(pub *rsa.PublicKey, hash crypto.Hash, hashed []byte, sig []byte) (out []byte, err error) {
293| hashLen, prefix, err := pkcs1v15HashInfo(hash, len(hashed))
294| if err != nil {
295| return nil, err
296| }
297|
298| tLen := len(prefix) + hashLen
299| k := (pub.N.BitLen() + 7) / 8
300| if k < tLen+11 {

```

```

301| return nil, fmt.Errorf("length illegal")
302| }
303|
304| c := new(big.Int).SetBytes(sig)
305| m := encrypt(new(big.Int), pub, c)
306| em := leftPad(m.Bytes(), k)
307| out = unLeftPad(em)
308|
309| err = nil
310| return
311|}
===== 文件: utils\bug.go =====
1|package utils
2|
3|import (
4| "reflect"
5| "runtime"
6|)
7|
8|// 获取当前包名 结构体名称,方法名称
9|func Q取包名结构体方法(结构体 interface{}) string {
10| // 使用反射获取当前包名
11| pkgPath := reflect.TypeOf(结构体).PkgPath()
12| //fmt.Println("当前包名:", pkgPath)
13|
14| // 使用反射获取结构体名称
15| structName := reflect.TypeOf(结构体).Name()
16| //fmt.Println("结构体名称:", structName)
17|
18| // 使用runtime包获取当前方法的名称
19| pc, _, _, _ := runtime.Caller(0)
20| funcName := runtime.FuncForPC(pc).Name()
21| //fmt.Println("方法名:", funcName)
22| return pkgPath + "-&gt;" + structName + "-&gt;" + funcName
23|}
===== 文件: utils\directory.go =====
1|package utils
2|
3|//目录 相关
4|import (
5| "errors"
6| "os"
7|)
8|
9|/*
10|@author: [piexlmax](https://github.com/piexlmax)
11|@function: PathExists
12|@description: 文件目录是否存在
13|@param: path string adwada
14|@return: bool, error
15|*/
16|func PathExists(path string) (bool, error) {
17| fi, err := os.Stat(path)
18| if err == nil {
19| if fi.IsDir() {
20| return true, nil
21| }
22| return false, errors.New("存在同名文件")
23| }
24| if os.IsNotExist(err) {

```

```

25| return false, nil
26| }
27| return false, err
28|}
29|
30|//@author: [piexlmax](https://github.com/piexlmax)
31|//@function: CreateDir
32|//@description: 批量创建文件夹
33|//@param: dirs ...string
34|//@return: err error
35|
36|func CreateDir(dirs ...string) (err error) {
37| for _, v := range dirs {
38| exist, err := PathExists(v)
39| if err != nil {
40| return err
41| }
42| if !exist {
43| if err := os.MkdirAll(v, os.ModePerm); err != nil {
44| return err
45| }
46| }
47| }
48| return err
49|}
==== 文件: utils\fmt_plus.go ====
1|package utils
2|
3|import (
4| "fmt"
5| "reflect"
6| "strings"
7|)
8|
9|//@author: [piexlmax](https://github.com/piexlmax)
10|//@function: StructToMap
11|//@description: 利用反射将结构体转化为map
12|//@param: obj interface{}
13|//@return: map[string]interface{}
14|
15|func StructToMap(obj interface{}) map[string]interface{} {
16| obj1 := reflect.TypeOf(obj)
17| obj2 := reflect.ValueOf(obj)
18|
19| data := make(map[string]interface{})
20| for i := 0; i < obj1.NumField(); i++ {
21| if obj1.Field(i).Tag.Get("mapstructure") != "" {
22| data[obj1.Field(i).Tag.Get("mapstructure")] = obj2.Field(i).Interface()
23| } else {
24| data[obj1.Field(i).Name] = obj2.Field(i).Interface()
25| }
26| }
27| return data
28|}
29|
30|//@author: [piexlmax](https://github.com/piexlmax)
31|//@function: ArrayToString
32|//@description: 将数组格式化为字符串
33|//@param: array []interface{}
34|//@return: string

```

```

35|
36|func ArrayToString(array []interface{}) string {
37| return strings.Replace(strings.Trim(fmt.Sprintf(array), ""), " ", "", -1)
38|}
==== 文件: utils\hash.go ====
1|package utils
2|
3|import (
4| "crypto/md5"
5| "fmt"
6| "strings"
7|)
8|
9|// BcryptHash 使用 bcrypt 对密码进行md5加密
10|func BcryptHash(password string) string {
11| bytes := Md5String(password)
12| return bytes
13|}
14|
15|// BcryptCheck 对比明文密码和数据库的哈希值
16|func BcryptCheck(password, Md5PassWord string) bool {
17| return strings.ToUpper(Md5String(password)) == strings.ToUpper(Md5PassWord)
18|}
19|}
20|func Md5String(str string) string {
21| data := []byte(str) //切片
22| has := md5.Sum(data)
23| md5str := fmt.Sprintf("%x", has) //将[]byte转成16进制
24| return md5str
25|}
==== 文件: utils\reload.go ====
1|package utils
2|
3|import (
4| "errors"
5| "os"
6| "os/exec"
7| "runtime"
8| "strconv"
9|)
10|
11|func Reload() error {
12| if runtime.GOOS == "windows" {
13| return errors.New("系统不支持")
14| }
15| pid := os.Getpid()
16| cmd := exec.Command("kill", "-1", strconv.Itoa(pid))
17| return cmd.Run()
18|}
19|
20|//func Reload热重启(继承端口 int) error {
21|// if runtime.GOOS == "windows" {
22|// return errors.New("不支持windows")
23|// }
24|//
25|// 新服务器 := endless.NewServer(":"+strconv.Itoa(继承端口), gin.Default())
26|// return 新服务器.ListenAndServe()
27|//
28|//}
==== 文件: utils\server.go ====

```

```

1|package utils
2|
3|import (
4| "github.com/shirou/gopsutil/v3/cpu"
5| "github.com/shirou/gopsutil/v3/disk"
6| "github.com/shirou/gopsutil/v3/mem"
7| "runtime"
8| "time"
9|)
10|
11|const (
12| B = 1
13| KB = 1024 * B
14| MB = 1024 * KB
15| GB = 1024 * MB
16|)
17|
18|type Server struct {
19| Os Os `json:"os"`
20| Cpu Cpu `json:"cpu"`
21| Ram Ram `json:"ram"`
22| Disk Disk `json:"disk"`
23|}
24|
25|type Os struct {
26| GOOS string `json:"goos"`
27| NumCPU int `json:"numCpu"`
28| Compiler string `json:"compiler"`
29| GoVersion string `json:"goVersion"`
30| NumGoroutine int `json:"numGoroutine"`
31|}
32|
33|type Cpu struct {
34| Cpus []float64 `json:"cpus"`
35| Cores int `json:"cores"`
36|}
37|
38|type Ram struct {
39| UsedMB int `json:"usedMb"`
40| TotalMB int `json:"totalMb"`
41| UsedPercent int `json:"usedPercent"`
42|}
43|
44|type Disk struct {
45| UsedMB int `json:"usedMb"`
46| UsedGB int `json:"usedGb"`
47| TotalMB int `json:"totalMb"`
48| TotalGB int `json:"totalGb"`
49| UsedPercent int `json:"usedPercent"`
50|}
51|
52|// @author: [SliverHorn](https://github.com/SliverHorn)
53|// @function: InitCPU
54|// @description: OS信息
55|// @return: o Os, err error
56|
57|func InitOS() (o Os) {
58| o.GOOS = runtime.GOOS
59| o.NumCPU = runtime.NumCPU()
60| o.Compiler = runtime.Compiler

```

```

61| o.GoVersion = runtime.Version()
62| o.NumGoroutine = runtime.NumGoroutine()
63| return o
64|}
65|
66|//@author: [SliverHorn](https://github.com/SliverHorn)
67|//@function: InitCPU
68|//@description: CPU信息
69|//@return: c Cpu, err error
70|
71|func InitCPU() (c Cpu, err error) {
72| if cores, err := cpu.Counts(false); err != nil {
73| return c, err
74| } else {
75| c.Cores = cores
76| }
77| if cpus, err := cpu.Percent(time.Duration(200)*time.Millisecond, true); err != nil {
78| return c, err
79| } else {
80| c.Cpus = cpus
81| }
82| return c, nil
83|}
84|
85|//@author: [SliverHorn](https://github.com/SliverHorn)
86|//@function: InitRAM
87|//@description: RAM信息
88|//@return: r Ram, err error
89|
90|func InitRAM() (r Ram, err error) {
91| if u, err := mem.VirtualMemory(); err != nil {
92| return r, err
93| } else {
94| r.UsedMB = int(u.Used) / MB
95| r.TotalMB = int(u.Total) / MB
96| r.UsedPercent = int(u.UsedPercent)
97| }
98| return r, nil
99|}
100|
101|//@author: [SliverHorn](https://github.com/SliverHorn)
102|//@function: InitDisk
103|//@description: 硬盘信息
104|//@return: d Disk, err error
105|
106|func InitDisk() (d Disk, err error) {
107| if u, err := disk.Usage("/"); err != nil {
108| return d, err
109| } else {
110| d.UsedMB = int(u.Used) / MB
111| d.UsedGB = int(u.Used) / GB
112| d.TotalMB = int(u.Total) / MB
113| d.TotalGB = int(u.Total) / GB
114| d.UsedPercent = int(u.UsedPercent)
115| }
116| return d, nil
117|}
==== 文件: utils\utils.go =====
1|// Package utils 公共工具
2|package utils

```

===== 文件: utils\工具测试_test.go =====

```
1|package utils
2|
3|import (
4| "crypto"
5| "crypto/md5"
6| "crypto/rand"
7| "crypto/rsa"
8| "crypto/x509"
9| "encoding/base64"
10| "encoding/hex"
11| "encoding/pem"
12| "fmt"
13| "server/utils/Qqwry"
14| "strconv"
15| "strings"
16| "testing"
17| "time"
18|)
19|
20|const 测试Rsa私钥 = `-----BEGIN RSA PRIVATE KEY-----
21|MIICWwIBAAKBgQCVp+zp4GDurEFIDGXu1gpLwgnpJCxQpv9WBhdIYwQcHVTOM4fH
22|A54nFP6EWNEBPXuqYRnkAcY0GFPPVg94tCTcGYhNM1YE1xdT20wCnHBigp6Ftp+2
23|LUDec73d5qqsSXYvuRj44c6sINxFlqpGsqwv/5GFKQA2DDRTFvgodMDSwwIDAQAB
24|AoGARDPKUW+TXVldX1q+HZFoEcX1Tc3RcFQa625NPURZvCJV4r8zMqXgZ/k2YMRf
25|Q+ZpPg4QiYuRc25e12kEFgMrBH0qWNebCF8iOnTx3VDC/0J6CeSRiktx0DEC7dTp
26|hBaXQeyitz3vMAGpgS50hc47x51Uq2YnrPeEp3LP4wLa4DECQQDEMw50W6eLa+14
27|acRM4i1Qqkv40XT2Xl/6yJPvuCqhRHEa0HSlx46baLz6u9AI42oWEW6yAtUZWKHI
28|Omr7toKLAKeAw0UzEn+FMdBLRtx/lazF4jzxGvGn0DmXKNCunoFbNrLF5XIQFlyW
29|xlQBfO42u1MkNFNdZxSqfTHOyshaWyuPqQJAUc2r4C363HHCKGsg8qF3HWPZzymh
30|57sPr6ODsvaJp7W+ptx1Vk2vEvMHoe9AfQQ8zji0e7ocmbXPtoIM4+iD4wJACnkc
31|qyJR4EveGZ9JU718hNorjiV0pi0LEk9BF1VwhJGkO7UNA7VO0mYQVhxUQy9hVzv
32|RocSODLBD5k9V2R3uQJAFxKUFRuGi4JmSPDhU8BqmoJWK4tuYypSi8uVlx0b6Suz
33|EGfbpm/BNVHVgtS5Afay//CDB03B2MbbsKerKPIhoQ==
34|-----END RSA PRIVATE KEY-----
35|`
36|const 测试Rsa公钥 = `-----BEGIN PUBLIC KEY-----
37|MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQCVp+zp4GDurEFIDGXu1gpLwgnp
38|JCxQpv9WBhdIYwQcHVTOM4fHA54nFP6EWNEBPXuqYRnkAcY0GFPPVg94tCTcGYhN
39|M1YE1xdT20wCnHBigp6Ftp+2LUDec73d5qqsSXYvuRj44c6sINxFlqpGsqwv/5GF
40|KQA2DDRTFvgodMDSwwIDAQAB
41|-----END PUBLIC KEY-----
42|`
43|
44|func Test_启动子程序(t *testing.T) {
45| //KuaiYanUpdater.B宝塔_修改项目信息()
46| Rsa加解密测试()
47| fmt.Println("执行完毕")
48|}
49|
50|func Ip查地址测试() {
51| ip地址 := "1.57.10.118" // 要查询的IP地址
52| 局_耗时 := time.Now().UnixMilli()
53| for i := 0; i < 10000; i++ {
54| _ _ = Qqwry.C查询IP归属地(ip地址)
55| }
56| fmt.Printf("1w次ip查询耗时:%dms\n", time.Now().UnixMilli()-局_耗时)
57|}
58|
59|func Aes加解密测试() {
```

```

60|
61| 局_耗时 := time.Now().UnixNano() / 1e6
62| for l := 0; l < 100000; l++ {
63| _ = Aes加密_cbc192("123456", "0NQ158xWiLJeLUCh8zhJI7ekpAZPvWIC")
64| }
65| 局_耗时 = time.Now().UnixNano()/1e6 - 局_耗时
66| fmt.Println("Aes加密耗时:" + strconv.FormatInt(局_耗时, 10))
67|
68| 密文 := Aes加密_cbc192("123465", "0NQ158xWiLJeLUCh8zhJI7ekpAZPvWIC")
69| fmt.Println(hex.EncodeToString(密文))
70| 明文 := Aes解密_cbc192(密文, "0NQ158xWiLJeLUCh8zhJI7ekpAZPvWIC")
71| fmt.Println(明文)
72| fmt.Println("签名_Aes")
73| fmt.Println(签名_Aes("OH+62zyTtaDXdnBjzwSmCYBG4JWsq1BtxqjQANVlarRLixwqJ9JPQNWErDfet5WtUGztsr484k3rfCkx
BZ6O2gli6SYCizahqxmOJqVRqMcDqJeHlusMA/cdO5j7hLQH8goiOXs6pD1s0HxZ73rGtAYPdQYB7P3nfaZMTRtcbq8=",
"0NQ158xWiLJeLUCh8zhJI7ekpAZPvWIC"))
74| fmt.Println("签名_Rsa")
75| fmt.Println(签名_Rsa("OH+62zyTtaDXdnBjzwSmCYBG4JWsq1BtxqjQANVlarRLixwqJ9JPQNWErDfet5WtUGztsr484k3rfCkx
BZ6O2gli6SYCizahqxmOJqVRqMcDqJeHlusMA/cdO5j7hLQH8goiOXs6pD1s0HxZ73rGtAYPdQYB7P3nfaZMTRtcbq8=",
测试Rsa私钥))
76|}
77|
78|func Rsa加解密测试() {
79| 局_明文 := "测试专用明文"
80| base64密文 := Rsa公钥加密(测试Rsa公钥, []byte(局_明文))
81| 局_密文字节集, _ := base64.StdEncoding.DecodeString(base64密文)
82| 局_明文 = Rsa私钥解密([]byte(测试Rsa私钥), 局_密文字节集)
83|
84| 局_明文 = "测试专用明文"
85| base64密文 = Rsa私钥签名(局_明文, 测试Rsa私钥)
86|
87| print(Rsa公钥验签(局_明文, base64密文, 测试Rsa公钥))
88|
89| 局_明文 = "测试专用明文"
90| base64密文 = RSA私钥加密([]byte(测试Rsa私钥), []byte(局_明文))
91| 局_密文字节集, _ = base64.StdEncoding.DecodeString(base64密文)
92| 局_明文 = string(RSA公钥解密(测试Rsa公钥, 局_密文字节集))
93|
94|}
95|
96|func Rsa测试() {
97| 错误, 公钥base64, 私钥base64 := GetRsaKey()
98|
99| fmt.Println("私钥:")
100| fmt.Println(私钥base64)
101| fmt.Println("公钥:")
102| fmt.Println(公钥base64)
103| fmt.Println(错误)
104|}
105|
106|func 签名_Aes(base64后明文 string, AesKey string) string {
107| return strings.ToUpper(Md5String(base64后明文 + AesKey))
108|}
109|
110|func 签名_Rsa(base64后明文 string, RSA私钥 string) string {
111|
112| pemKey := []byte(RSA私钥)
113|
114| data := []byte(base64后明文)
115| hashMd5 := md5.Sum(data)

```

```

116| hashed := hashMd5[:]
117| block, _ := pem.Decode(pemKey)
118| if block == nil {
119| return ""
120| }
121| privateKey, err := x509.ParsePKCS1PrivateKey(block.Bytes) //大坑 必须校验是否 block是否为nil
否则block.Bytes卡主,类似进入许可区,
122| if err != nil {
123| return ""
124| }
125| signature, err := rsa.SignPKCS1v15(rand.Reader, privateKey, crypto.MD5, hashed)
126| return strings.ToUpper(hex.EncodeToString(signature))
127|}
===== 文件: utils\正则.go =====
1|package utils
2|
3|import (
4| "regexp"
5| "strconv"
6|)
7|
8|// 校验密码 "至少八个字符, 至少一个字母和一个数字: "
9|func Z正则_校验密码(s string, msg *string) bool {
10| 匹配结果, _ := regexp.MatchString(`\S{5,18}$`, s)
11|  //fmt.Println("Z正则_校验密码$s", a)
12|
13| if !匹配结果 {
14|  *msg = "长度在5-18之间, 只能包含字母、数字和下划线等非空白字符"
15| }
16| return 匹配结果
17|}
18|
19|func Z正则_校验代理用户名(s string, msg *string) bool {
20| 匹配结果, _ := regexp.MatchString(`^[a-zA-Z0-9\p{Han}]+$`, s)
21| if !匹配结果 {
22|  *msg = "只能包含英文字母、数字和ANSI编码支持的中文字符"
23| }
24| return 匹配结果
25|}
26|
27|// Z正则_校验用户名 "至少6个字符,支持数字大小写字母"
28|func Z正则_校验用户名(s string, msg *string) bool {
29| 匹配结果, _ := regexp.MatchString(`\w{5,17}$`, s)
30| if 匹配结果 {
31| return 匹配结果
32| }
33| *msg = "长度在6-18之间, 只能包含字符、数字和下划线"
34| return 匹配结果
35|}
36|
37|// Z正则_校验email "非正确email格式"
38|func Z正则_校验email(s string, msg *string) bool {
39| 匹配结果, _ := regexp.MatchString(`^\w+([-+.]w+)*@\w+([-.]w+)*\.\w+([-.]w+)*$`, s)
40| if 匹配结果 {
41| return 匹配结果
42| }
43| *msg = "非正确email格式"
44| return 匹配结果
45|}
46|

```

```

47|// Z正则_校验纯数字 "非完全是数字"
48|func Z正则_校验纯数字(s string, msg *string) bool {
49| 匹配结果, _ := regexp.MatchString(`^-\?d*\.\?\d+$`, s)
50|  if 匹配结果 {
51|    return 匹配结果
52|  }
53|  *msg = "非完全是数字"
54|  return 匹配结果
55|}
56|
57|// Z正则_校验纯数字指定位数 "长度不为"+位数
58|func Z正则_校验纯数字指定位数(s string, msg *string, 位数 int) bool {
59| 匹配结果, _ := regexp.MatchString(`^d{`+strconv.Itoa(位数)+`}$`, s)
60|
61|  if 匹配结果 {
62|    return 匹配结果
63|  }
64|  *msg = "长度不为" + strconv.Itoa(位数)
65|  return 匹配结果
66|}
67|
68|// Z正则_校验手机号 "非正确手机号格式"
69|func Z正则_校验手机号(s string, msg *string) bool {
70| 匹配结果, _ := regexp.MatchString(`1[3,4,5,6,7,8,9]d[s,-]?\d{4}[s,-]?\d{4}`, s)
71|  if 匹配结果 {
72|    return 匹配结果
73|  }
74|  *msg = "非正确手机号格式"
75|  return 匹配结果
76|}
77|
78|// Z正则_是否英数
79|func Z正则_是否英数(s string, msg *string) bool {
80|  // 定义正则表达式
81|  reg := regexp.MustCompile(`^[A-Za-z0-9]+$`)
82|  // 使用正则表达式进行匹配
83|  if reg.MatchString(s) {
84|    return true
85|  }
86|  *msg = "只能输入数字字母"
87|  return false
88|}
===== 文件: utils\系统.go =====
1|package utils
2|
3|import (
4|  "EFunc/utils"
5|  "fmt"
6|  "log"
7|  "os"
8|  "path/filepath"
9|  "strings"
10|  "time"
11|)
12|
13|func X系统_权限检测() bool {
14|  //检查自身进程是否有写入数据的权限
15|  // 这个例子测试写权限, 如果没有写权限则返回error。
16|  // 注意文件不存在也会返回error, 需要检查error的信息来获取到底是哪个错误导致。
17|

```

```
18| utils.W文件_删除(GetCurrentAbPathByExecutable() + "/权限测试.json")
19|
20| f, err := os.Create(GetCurrentAbPathByExecutable() + "/权限测试.json")
21| if err != nil {
22| return false
23| }
24| defer f.Close()
25| f.WriteString(time.Now().String())
26| // 获取当前权限
27| fi, err := f.Stat()
28| if err != nil {
29| return false
30| }
31| fmt.Printf("文件权限 %v\n", fi.Mode())
32| return true
33|}
34|
35|// 获取当前执行程序所在的绝对路径
36|func GetCurrentAbPathByExecutable() string {
37| exePath, err := os.Executable()
38| if err != nil {
39| log.Fatal(err)
40| }
41| res, _ := filepath.EvalSymlinks(filepath.Dir(exePath))
42| return res
43|}
44|
45|func Y易支付取设备类型(ua string) string {
46| // pc 电脑浏览器
47| // mobile 手机浏览器
48| // qq 手机QQ内浏览器
49| // wechat 微信内浏览器
50| // alipay 支付宝客户端
51| // jump 仅返回支付跳转url
52|
53| // 转换为小写以便匹配
54| ua = strings.ToLower(ua)
55|
56| // 定义各类客户端的关键词
57| mobileKeywords := []string{
58| "mobile", "android", "iphone", "ipod", "ipad", "windows phone",
59| }
60|
61| wechatKeywords := []string{
62| "micromessenger",
63| }
64|
65| qqKeywords := []string{
66| "qq/", "qzone/",
67| }
68|
69| alipayKeywords := []string{
70| "alipayclient",
71| }
72|
73| // 检查是否为支付宝客户端
74| for _, keyword := range alipayKeywords {
75| if strings.Contains(ua, keyword) {
76| return "alipay" // alipay
77| }
```

```
78| }
79|
80| // 检查是否为微信浏览器
81| for _, keyword := range wechatKeywords {
82| if strings.Contains(ua, keyword) {
83| return "wechat" // wechat
84| }
85| }
86|
87| // 检查是否为QQ浏览器
88| for _, keyword := range qqKeywords {
89| if strings.Contains(ua, keyword) {
90| return "qq" // qq
91| }
92| }
93|
94| // 检查是否为移动设备
95| for _, keyword := range mobileKeywords {
96| if strings.Contains(ua, keyword) {
97| return "mobile" // mobile
98| }
99| }
100|
101| // 默认返回false, 表示PC或其他未识别设备
102| return "pc"
103|}
```